
Chapter 13 Searching and Sorting

13.1 Searching and Sorting in the Java Class Libraries

- Binary Search 
- Sorting 
- Shuffling 
- Custom Ordering with Comparators 

13.2 Program Complexity

- Empirical Analysis 
- Complexity Classes 

13.3 Implementing Searching and Sorting Algorithms

- Sequential Search 
- Binary Search 
- Recursive Binary Search 
- Searching Objects 
- Selection Sort 

13.4 Case Study: Implementing Merge Sort

- [Splitting and Merging Arrays !\[\]\(467d80e979964f7f8c752fb22248b5b7_img.jpg\)](#)
- [Recursive Merge Sort !\[\]\(b71552d33dbf62adf5e5199a70ee02bf_img.jpg\)](#)
- [Complete Program !\[\]\(03134b765d1473836ff001925b1b0550_img.jpg\)](#)

Introduction

When you are dealing with large amounts of data, you'll often want to search the data for particular values. For example, you might want to search the Internet for a web page that contains a certain keyword, or you might want to search a phone book for a person's phone number.

It's also useful to be able to rearrange a collection of data into sorted order. For example, you might want to sort a list of students' course grade data by name, student ID, or grade.

In this chapter we'll look at ways to use Java's class libraries to search and sort data. We'll practice implementing some searching and sorting algorithms and talk more generally about

how to observe and analyze the runtimes of algorithms.

13.1 Searching and Sorting in the Java Class Libraries

To search an `ArrayList` or `LinkedList`, you can call its `indexOf` method. This method examines each element of the list, looking for a target value. It returns the first index at which the target value occurs in the list, or `-1` if the target doesn't occur.

Imagine a large `List` containing all the words in a book. The following code could read a large text file into such a list:

```
// reads the text of the given file into a list
public static List<String> readBook(String filename)
    throws FileNotFoundException {
    List<String> words = new ArrayList<>();
    Scanner in = new Scanner(new File(filename));
    while (in.hasNext()) {
        words.add(in.next());
    }
    return words;
}
```

You could use the `indexOf` method to see whether a given word appears in the book and, if so, at what index the word appears:

```
System.out.print("Your word? ");  
Scanner console = new Scanner(System.in);  
String word = console.nextLine();  
// search list for a word using indexOf  
List<String> words = readBook("mobydick.txt");  
int index = words.indexOf(word);  
if (index >= 0) {  
    System.out.println(word + " is word #" + index);  
} else {  
    System.out.println(word + " is not found.");  
}
```

The `indexOf` method performs a *sequential search*, examining each element of the list in sequence until it finds the one that the user is looking for. If it reaches the end of the list without finding the requested word, it returns `-1`. When it searches a 1,000,000-element list for an element at index 675,000, a sequential search would have to examine all 675,000 elements that came before the desired element.

If you have an array of data instead of a list, there's no prewritten method to sequentially search the array. You'll have to write the code

yourself (as we'll do later in this chapter) or put the array's elements into a `List` first and search the `List` with `indexOf`.

Binary Search



Sometimes you'll want to search through elements of an array or list that you know is in sorted order. For example, if you wanted to know whether “queasy” was a real English word, you could search the contents of an alphabetized dictionary text file. Likewise, you might find yourself looking for a book written by Robert Louis Stevenson in a list of books sorted by the author's last name. If the dictionary is large or the list of books is long, you probably won't want to sequentially examine all the items it contains.

There's a better algorithm called *binary search* that searches sorted data much faster than a sequential search. A normal sequential search of a million-element array may have to examine all the elements, but a binary search will need to look at only around 20 of

them. Java's class libraries contain methods that implement the binary search algorithm for arrays and lists.

Binary Search

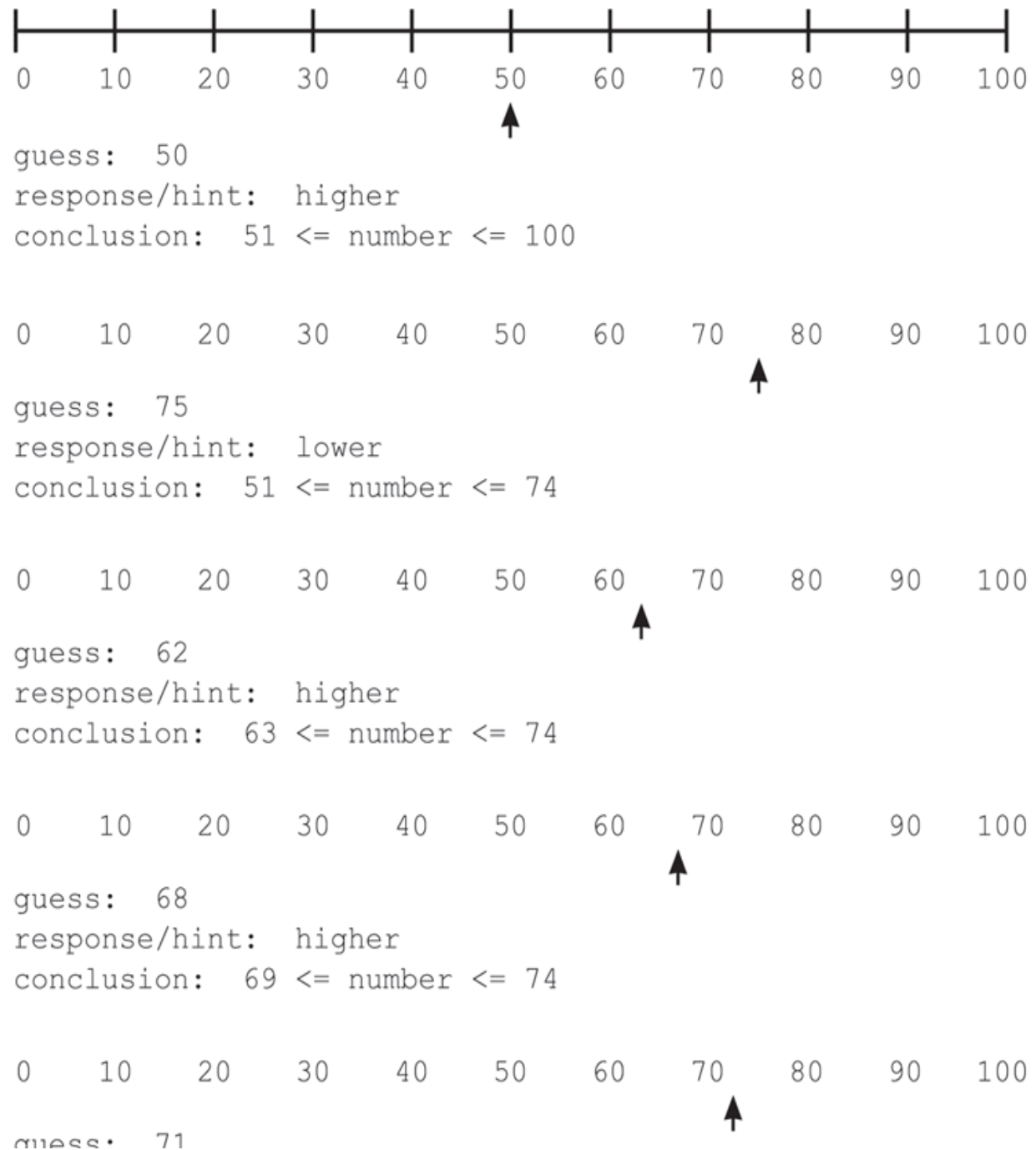
An algorithm that searches for a value in a sorted list by repeatedly dividing the search space in half.

The binary search algorithm begins by examining the center element of the array or list. If the center element is smaller than the target you're searching for, there's no reason to examine any elements to the left of the center (at lower indexes). If the center element is larger than the target you're searching for, there's no reason to examine any elements to the right of the center (at greater indexes). Each pass of the algorithm eliminates half the search space from consideration, so in most cases the algorithm finds the target value much faster than a sequential search would have found it.

The logic of the binary search algorithm is similar to the strategy that people use in a high/low guessing game in which the computer generates a random number between 1 and 100 and the user tries to guess it. After each incorrect guess, the program gives a hint about whether the user's guess was too high or too low. A poor algorithm for this game is to guess 1, 2, 3, and so on. A smarter algorithm is to

guess the middle number and cut the range in half each time on the basis of whether the guess was too high or too low. **Figure 13.1** shows how this works.

Figure 13.1 Passes of a binary search for a number between 1 and 100



```
guess: 71  
response/hint: higher  
conclusion: 72 <= number <= 74
```

```
0    10    20    30    40    50    60    70    80    90   100  
                                     ↑  
guess: 73  
response/hint: correct!
```

A binary search uses this same approach when it searches a sorted array for a target value. The algorithm scales extremely well to large input data. The `Arrays` class in the `java.util` package contains a static method called `binarySearch` that implements the binary search algorithm. It accepts an array of any suitable type and a target value as its parameters and returns the index where you can find the target element. If the element isn't found, the method returns a negative index.

The following code uses the `Arrays.binarySearch` method to find a number in an array of integers. It needs to examine only indexes 4, 6, and then 5 in order to find the target value at index 5:

```
// binary search on an array  
int[] numbers = {-3, 2, 8, 12, 17, 29, 44, 58, 79};  
int index = Arrays.binarySearch(numbers, 29);  
System.out.println("29 is found at index " + index);
```

If you're using a list such as an `ArrayList` instead, you can call the static method `Collections.binarySearch` to search the list of elements. If you had an `ArrayList` called `list` containing the same elements as the array in the previous example, the following code would similarly search the elements at indexes 4, 6, and then 5 before assigning the value `5` to the variable `index`:

```
// binary search on an ArrayList
int index = Collections.binarySearch(list, 29);
System.out.println("29 is found at index " + index);
```

If you want to use the `binarySearch` method on either an array or a list, the data must be in sorted order, because the method relies on the ordering to quickly find the target value. If you call `binarySearch` on unsorted data, the results are undefined and the algorithm doesn't guarantee that it will return the right answer.

Let's look at a short program that benefits from the speed of binary search. In the game of Scrabble, players form words on a board using letter tiles to earn points. Sometimes a player tries to spell a word that is not a legal word in the dictionary, so another player "challenges" the word. The challenger looks up the word in the dictionary, and depending on whether it is found, the move may be revoked from the board. The following program helps resolve Scrabble word challenges by performing binary searches for words

in a dictionary file. The input file's words occur in sorted order, so the list can be properly searched using `Collections.binarySearch`. (If the file had not been sorted, we could have sorted it with `Collections.sort`).

```
1 // Searches for words in a dictionary text file
2 // and reports each word's position in the file.
3
4 import java.io.*;
5 import java.util.*;
6
7 public class WordChallenge {
8     public static void main(String[] args)
9         throws FileNotFoundException {
10         System.out.println("Welcome to Scrabble word
challenge!");
11
12         // read a sorted dictionary file into a List
13         Scanner in = new Scanner(new File("words.txt"));
14         List<String> words = new ArrayList<>();
15         while (in.hasNext()) {
16             String word = in.next();
17             words.add(word);
18         }
19
20         // binary search the list for words
```

```

21         Scanner console = new Scanner(System.in);
22         System.out.print("Word to challenge (Enter to quit)?
");
23         String target = console.nextLine();
24         while (target.length() > 0) {
25             int index = Collections.binarySearch(words,
target);
26             if (index >= 0) {
27                 System.out.println "\"" + target + "\" is
word #"
28                     + index + " of " + words.size());
29             } else {
30                 System.out.println "\"" + target + "\" is
not found");
31             }
32
33             System.out.print("Word to challenge (Enter to
quit)? ");
34             target = console.nextLine();
35         }
36     }
37 }

```

Here is a sample execution of the program and its resulting output, using a Scrabble players' dictionary that contains 172,823 words:

```
Welcome to Scrabble word challenge!
Word to challenge (Enter to quit)? queazy
"queazy" is word #121788 of 172823
Word to challenge (Enter to quit)? kwyjibo
"kwyjibo" is not found
Word to challenge (Enter to quit)? building
"building" is word #18823 of 172823
Word to challenge (Enter to quit)? java
"java" is word #79156 of 172823
Word to challenge (Enter to quit)? programs
"programs" is word #118860 of 172823
Word to challenge (Enter to quit)?
```

Sorting



When you use a computer, you often need to sort data. When you browse your hard drive, for example, you might sort your files by file name, extension, and date. When you play music, you might sort your song collection by artist, year, or genre. You might also want to sort arrays and lists so that they can be searched efficiently with the binary search algorithm.

The Java class libraries provide sorting methods for arrays and lists. You can sort an array with the `Arrays.sort` method:

```
// demonstrate the Arrays.sort method
String[] strings = {"c", "b", "g", "h", "d", "f", "e", "a"};
Arrays.sort(strings);
System.out.println(Arrays.toString(strings));
```

The preceding code produces the following output:

```
[a, b, c, d, e, f, g, h]
```

Java 8 introduced a new sorting method called `Arrays.parallelSort` that generally runs faster than `Arrays.sort`. It takes advantage of computers with multiple processors or multi-core processors to sort multiple portions of the array at the same time, leading to faster

performance. To use it, just write `parallelSort` instead of `sort` in your code. Other new features of Java 8 are discussed in more detail in [Chapter 19](#).

```
Arrays.parallelSort(strings);
```

The array must be of a type that can be compared—that is, a type that stores either primitives or objects that implement the `Comparable` interface discussed in [Chapter 10](#). For example, you can sort an array of integers or `Strings`, but you can't easily sort an array of `Point` objects or `Color` objects because those classes don't implement the `Comparable` interface.

To sort a list you can use the `Collections.sort` method, discussed briefly in [Chapter 10](#), which accepts a list such as an `ArrayList` as a parameter and puts its elements into sorted order. The following code produces the same output as the preceding array code:

```
// demonstrate the Collections.sort method
List<String> list = new ArrayList<>();
list.add("c");
list.add("b");
list.add("g");
list.add("h");
```

```
list.add("d");  
list.add("f");  
list.add("e");  
list.add("a");  
  
Collections.sort(list);  
  
System.out.println(list);
```

When the `Arrays.sort` method is used with primitive data, it uses an algorithm called quicksort. `Collections.sort` and `Arrays.sort` use a different algorithm, called merge sort, when they deal with object data. We'll discuss the implementation of merge sort in detail later in this chapter.

Shuffling

The task of shuffling data, or rearranging the elements into a random order, is perhaps the opposite of sorting. Why would one want to do this?

One application for shuffling is a card game program. You might have a card deck stored as a list of `Card` objects. If the cards are in a predictable order, the game will be boring. You'd like to shuffle the deck of cards, rearranging them into a random ordering each time. This is a case in which chaos is preferable to order.

Another application is a situation in which you want a random permutation of a list of numbers. You can acquire a random permutation of the numbers from 1 through 5, for example, by storing those numbers into a list and shuffling the list.

The `Collections` class has a method called `shuffle` that accepts a list as its parameter and rearranges its elements randomly. The following example creates a deck of card strings, shuffles it, and examines the card at the top of the deck:

```
String[] ranks = {"2", "3", "4", "5", "6", "7", "8", "9",  
                 "10", "Jack", "Queen", "King", "Ace"};  
String[] suits = {"Clubs", "Diamonds", "Hearts", "Spades"};  
List<String> deck = new ArrayList<>();  
for (String rank : ranks) { // build sorted deck  
    for (String suit : suits) {  
        deck.add(rank + " of " + suit);  
    }  
}  
  
Collections.shuffle(deck);  
  
System.out.println("Top card = " + deck.get(0));
```

The code randomly produces output such as the following, with different outputs on different runs:

```
Top card = 10 of Spades
```

Table 13.1  briefly summarizes the useful static methods in Java's class libraries for searching, sorting, and shuffling.

Table 13.1 Searching and Sorting in Java's Class Libraries

Custom Ordering with Comparators

Sometimes you'll want to search or sort a collection of objects in an ordering that is different from that of its `Comparable` implementation. For example, consider the following code, which sorts an array of `Strings` and prints the result:

```
String[] strings = {"Foxtrot", "alpha", "echo", "golf",  
                    "bravo", "hotel", "Charlie", "DELTA"};  
Arrays.sort(strings);  
System.out.println(Arrays.toString(strings));
```

This code produces the following output, which may not be what you expected:

```
[Charlie, DELTA, Foxtrot, alpha, bravo, echo, golf, hotel]
```

Notice that the elements are in case-sensitive alphabetical ordering, with all the uppercase strings placed before the lowercase ones.

Recall from [Chapter 10](#)  that many types (including `String`) have natural orderings that are defined by comparison functions, implemented as the `compareTo` method from the `Comparable` interface. The `compareTo` method of `String` objects uses a case-sensitive ordering, but in this case we would prefer a case-insensitive comparison. We might also want an option to sort in other orders, such as by length or in reverse alphabetical order.

In situations such as these, you can define your own external comparison functions with an object called a *comparator*. Comparators perform comparisons between pairs of objects. While a class can have only one natural ordering, arbitrarily many comparators can be written to describe other ways to order the class's objects. A comparator can also supply an external ordering on a class that does not implement a natural ordering of its own. The `Arrays.sort`, `Collections.sort`, and `Arrays.binarySearch` methods all have variations that accept a comparator as an additional parameter and use the comparator to guide the searching or sorting process.

Comparators are implemented as classes that implement the interface `Comparator` in the `java.util` package. This interface has a method called `compare` that accepts a pair of objects as parameters and compares them. Like `compareTo`, the `compare` method returns a negative number, zero, or a positive number to indicate a less-than, equal-to, or greater-than relationship, respectively:

```
public interface Comparator<T> {  
    public int compare(T o1, T o2);  
}
```

Implementing `Comparator` is like implementing the `Comparable` interface, except that instead of placing the code inside the class to be compared, you write a separate comparator class. The `compare` method is similar to the `compareTo` method from the `Comparable` interface, except that it accepts both of the objects to compare as parameters. (The `compareTo` method accepts only one object as a parameter because the other object compared is the implicit parameter.)

Like `Comparable`, `Comparator` is actually a generic interface `Comparator<T>` that requires you to specify the type of objects you'll be comparing. For example, if you wish to write a comparison function for `Strings`, you must write a class that implements `Comparator<String>`.

Before we implement a comparator of our own, let's use one that is part of Java's class libraries. If you want to sort `Strings` in case-insensitive alphabetical order, you can use a constant `Comparator` in the `String` class called `CASE_INSENSITIVE_ORDER`. The following code uses it to sort an array of mixed-case strings:

```
// sort Strings using case-insensitive Comparator
String[] strings = {"Foxtrot", "alpha", "echo", "golf",
                    "bravo", "hotel", "Charlie", "DELTA"};
Arrays.sort(strings, String.CASE_INSENSITIVE_ORDER);
System.out.println(Arrays.toString(strings));
```

This code produces the following output:

```
[alpha, bravo, Charlie, DELTA, echo, Foxtrot, golf, hotel]
```

Now let's write a `Comparator` that orders strings by their lengths. Our `compare` method should return a negative number if the first string is shorter than the second, `0` if they are equal in length, and a positive number if the first string is longer than the second. Recall from [Chapter 10](#)  that a `compareTo` method examining integer data can often simply subtract one number from the other and return the result. The following `compare` method is also based on this idea:

```

1  import java.util.*;
2
3  // compares String objects by length
4  public class LengthComparator implements Comparator<String>
5  {
6      public int compare(String s1, String s2) {
7          return s1.length() - s2.length();
8      }
9  }

```

Now that we've written a length comparator, we can pass one when we sort an array or list of `String` objects:

```

// sort array of strings by length using Comparator
Arrays.sort(strings, new LengthComparator());
System.out.println(Arrays.toString(strings));

```

Here's the output when we run this code on the `String` array from earlier in this section:

```
[echo, golf, alpha, bravo, hotel, DELTA, Foxtrot, Charlie]
```

Notice that the strings appear in order of increasing length.

Sometimes you'll want to search or sort a collection of objects that don't implement the `Comparable` interface. For example, the `Point` class doesn't implement `Comparable`, but you might want to sort an array of `Point` objects by x -coordinate, using the y -coordinate to break ties. The following code is an example `Comparator` that compares `Point` objects in this way:

```
1  import java.awt.*;
2  import java.util.*;
3
4  // compares Point objects by x-coordinate and then by y-
coordinate
5  public class PointComparator implements Comparator<Point> {
6      public int compare(Point p1, Point p2) {
7          int dx = p1.x - p2.x;
8          if (dx == 0) {
9              int dy = p1.y - p2.y;
10             return dy;
11         } else {
12             return dx;
13         }
14     }
15 }
```

It's important to understand that we are creating a separate class as our comparator; we are not modifying the `Point` class itself. This is one of the main benefits of comparators: you don't need to modify the class being ordered. The following code uses the `PointComparator` to sort an array of four `Point` objects:

```
Point[] points = {  
    new Point(4, -2),  
    new Point(3, 9),  
    new Point(-1, 15),  
    new Point(3, 7)  
};  
  
Arrays.sort(points, new PointComparator());
```

This code sorts the points into the following order: $(-1, 15)$, $(3, 7)$, $(3, 9)$, $(4, -2)$. This comparator also allows you to use `Point` objects in the “tree” collections that depend on ordering, `TreeSet` and `TreeMap`. Both of these collections have constructors that accept a comparator as a parameter and use it to order their elements internally.

If you have a group of objects with a natural ordering and you want to sort them in reverse order, you can call the method `Collections.reverseOrder` to receive a `Comparator` that inverts the objects' natural ordering. For example, the following code would sort

the array of `Strings` that was used earlier in reverse alphabetical order:

```
Arrays.sort(strings, Collections.reverseOrder());
```

For types that do not have a natural ordering, you can ask for the reverse of a `Comparator`'s order by calling `Collections.reverseOrder` and passing the `Comparator` to be reversed. For example, the following code would sort the array of `Point` objects by decreasing x -coordinate:

```
Arrays.sort(points, Collections.reverseOrder(new  
PointComparator()));
```

The resulting order would be as follows: (4, -2), (3, 9), (3, 7), (-1, 15).

Table 13.2  summarizes several useful places that comparators appear in the Java class libraries.

Table 13.2 Useful Comparators and Methods

13.2 Program Complexity



In [Chapter 8](#), we talked about client code, the code that interacts with a class or object. In this chapter, we've studied how to be a client of Java's methods for searching and sorting data. Since searching and sorting are important programming ideas, it's worthwhile to understand how they are implemented. But before we dive into this, let's discuss some background ideas about how to analyze the efficiency of code.

As you progress in this textbook, you're writing increasingly complex programs. You're also seeing that there are often many ways to solve the same problem. How do you compare different solutions to the same problem to see which is better?

We desire algorithms that solve problems quickly or with high efficiency. The technical term that refers to algorithms' runtime is

complexity. An algorithm with higher complexity uses more time or resources to solve a problem.

Complexity

A measure of the computing resources that are used by a piece of code, such as time, memory, or disk space.

Usually when we talk about the efficiency of a program we are talking about how long the program takes to run, or its *time complexity*. The time complexity for a program to be “fast enough” depends on the task. A program running on a modern computer that requires five minutes to look up a dictionary word is probably too slow. An algorithm that renders a complex three-dimensional movie scene in five minutes is probably very fast.

One way to determine an algorithm’s approximate time complexity is to program it, run the program, and measure how long it takes to run. This is sometimes called an *empirical analysis* of the algorithm. For example, consider two algorithms to search an array: one that sequentially searches for the desired target element, and one that first sorts the array and then performs a binary search on the sorted

array. You could empirically analyze the algorithms by writing both as programs, running them on the same input, and timing them.

But empirically analyzing an algorithm isn't a very reliable measure, because on a different computer with a different processor speed and more or less memory the program might not take the same amount of time. Also, in order to empirically test an algorithm, you must write it and time it, which can be a chore.

A more neutral way to measure a program's performance is to examine its code or pseudocode and roughly count the number of statements that are executed. This is a form of *algorithm analysis*, the practice of applying techniques to mathematically approximate the performance of various computing algorithms. Algorithm analysis is an important tool in computer science. One of the fundamental principles of science in general is that we can make predictions and hypotheses using formal models, which we can then test by experimentation.

Not all statements require the same amount of time to execute. For example, a CPU can handle addition faster than multiplication, and a method call generally takes more time than a statement that evaluates the Boolean test of an `if/else` statement. But for the purposes of simplification, let's assume that the following actions require an equal and fixed amount of time to execute:

- Variable declarations and assignments
- Evaluating mathematical and logical expressions

- Accessing or modifying an individual element of an array
- Simple method calls (where the method does not perform a loop)

One kind of variable that does not require a fixed amount of time to initialize is an array. When an array is constructed, Java zeroes each array element, which takes more time for longer arrays. Some types of objects also have lengthy code in their constructors that makes them take longer to construct.

From the preceding simple rules, we can extrapolate the runtimes of larger and more complex pieces of code. For example, the runtime of a group of statements in sequential order is the sum of the individual runtimes of the statements:

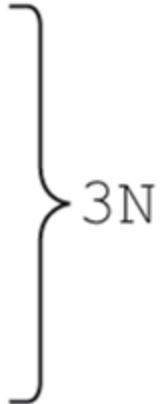
```
statement1. }  
statement2. } 3  
statement3. }
```

The runtime of a loop is roughly equal to the runtime of its body times the number of iterations of the loop. For example, a loop with a body that contains K simple statements and that repeats N times will have a runtime of roughly $(K * N)$:

```

for (N times) {
    statement1.
    statement2.
    statement3.
}

```



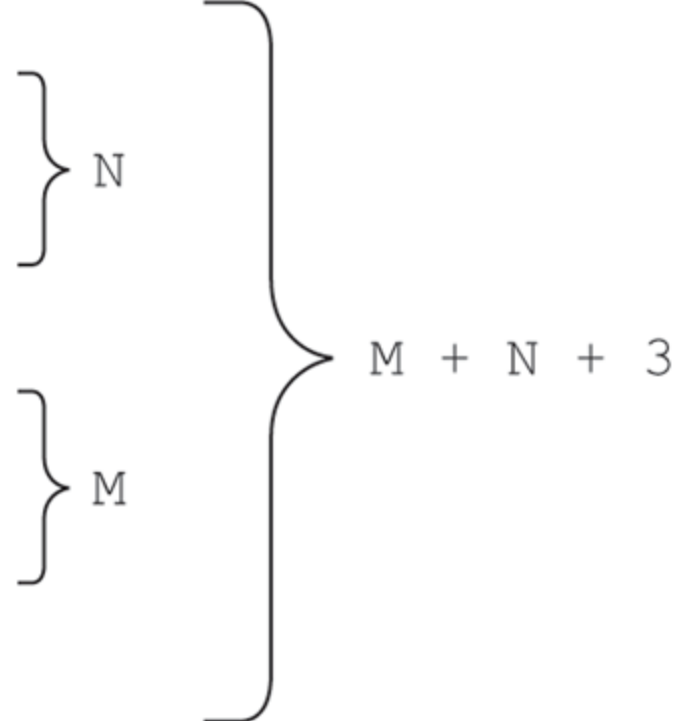
The runtime of multiple loops placed sequentially (not nested) with other statements is the sum of the loops' runtimes and the other statements' runtimes:

```

statement1.
for (N times) {
    statement2.
}

for (M times) {
    statement3.
}
statement4.
statement5.

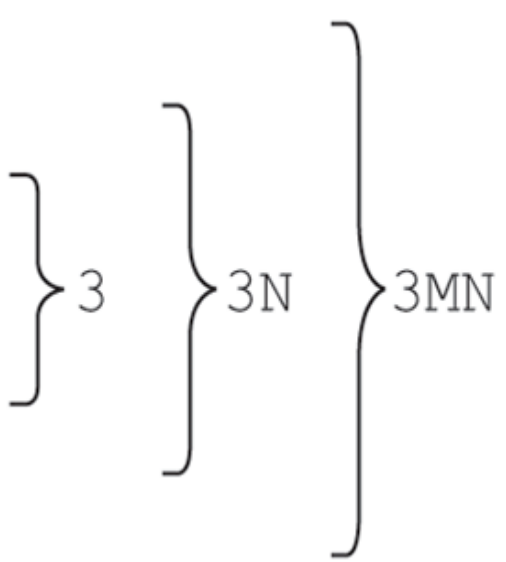
```



The runtime of a loop containing a nested loop is roughly equal to the runtime of the inner loop multiplied by the number of repetitions

of the outer loop:

```
for (M times) {  
    for (N times) {  
        statement1.  
        statement2.  
        statement3.  
    }  
}
```



Normally, the loops in long-running algorithms are processing some kind of data. Many algorithms run very quickly if the input dataset is small, so we generally worry about the performance only for large datasets. For example, consider the following set of loops that process an array of N elements:

```

for (N times) {
    for (N times) {
        statement1.
    }
}

for (N times) {
    statement2.
    statement3.
}

```

Diagram illustrating the execution counts for the code blocks:

- The first block (nested loops) is annotated with a brace indicating N^2 executions.
- The second block (sequential statements) is annotated with a brace indicating $2N$ executions.
- A large brace on the right groups both blocks, indicating a total execution count of $N^2 + 2N$.

When we analyze code like this, we often think about which line is most frequently executed in the code. In programs with several sequential blocks of code that all relate to some common value N (such as the size of an input dataset), the block raised to the highest power of N usually dominates the overall runtime. In the preceding code, the first N^2 loop executes its statement far more times than the second N loop executes its two statements. For example, if N is 1000, `statement1` executes $(1000 * 1000)$ or 1,000,000 times, while `statement2` and `statement3` each execute only 1000 times.

When we perform algorithm analysis, we often ignore all but the most frequently executed part of the code, because the runtime of this statement will outweigh the combined runtimes of the other parts of the code. For example, we might refer to the preceding code as being “on the order of” N^2 , ignoring the extra $2N$ statements altogether. We’ll revisit this idea later in the chapter.

One key concept to take away from this brief discussion of algorithm analysis is how expensive it is to perform nested loops over large sets of input data. Algorithms that make many nested passes over a very large dataset tend to perform poorly, so it's important to come up with efficient algorithms that don't loop over data needlessly.

Now let's take a look at algorithm complexity in action, observing the runtimes of some actual algorithms that can be used to solve a programming problem on a large dataset.

Empirical Analysis

Consider the task of computing the range of numbers in an array. The range is the difference between the lowest and highest numbers in the array. An initial solution might use nested loops to examine every pair of elements in the array, computing their difference and remembering the largest difference found:

```
max = 0.  
for (each index i) {  
    for (each index j) {  
        update max, if elements i and j differ by more than  
max.  
    }  
}
```

The following code implements the `range` method as described:

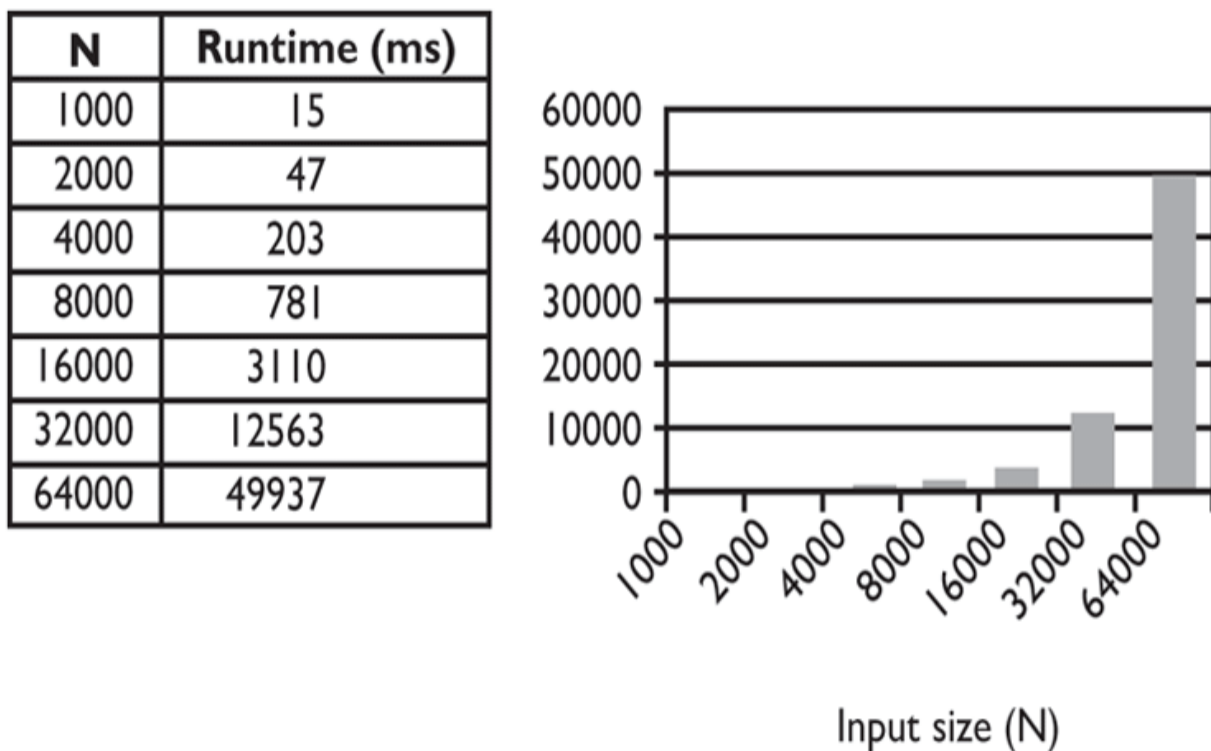
```
// returns the range of numbers in the given array
public static int range(int[] numbers) {
    int maxDiff = 0;
    for (int i = 0; i < numbers.length; i++) {
        for (int j = 0; j < numbers.length; j++) {
            int diff = Math.abs(numbers[j] - numbers[i]);
            maxDiff = Math.max(maxDiff, diff);
        }
    }
    return maxDiff;
}
```

Since the code has two nested `for` loops, each of which processes the entire array, we can hypothesize that the algorithm executes roughly N^2 statements, or some multiple thereof.

We can measure the speed of this range algorithm in milliseconds by calling `range` on various arrays and measuring the time elapsed. We measure the time by acquiring the current time before and after calling `range` on a large array and subtracting the start time from the end time.

As you can see in [Figure 13.2](#), as the input size N doubles, the runtime of the `range` method approximately quadruples. This is consistent with our hypothesis. If the algorithm takes N^2 statements to run and we increase the input size to $2N$, the new runtime is roughly $(2N)^2$ or $4N^2$, which is four times as long as the original runtime.

Figure 13.2 Runtimes for first version of range algorithm



Our code isn't very efficient for a large array. It requires over 12 seconds to examine 32,000 integers on a modern computer. In real-world data-processing situations, we would expect to see far larger input datasets than this, so this runtime isn't acceptable for general use.

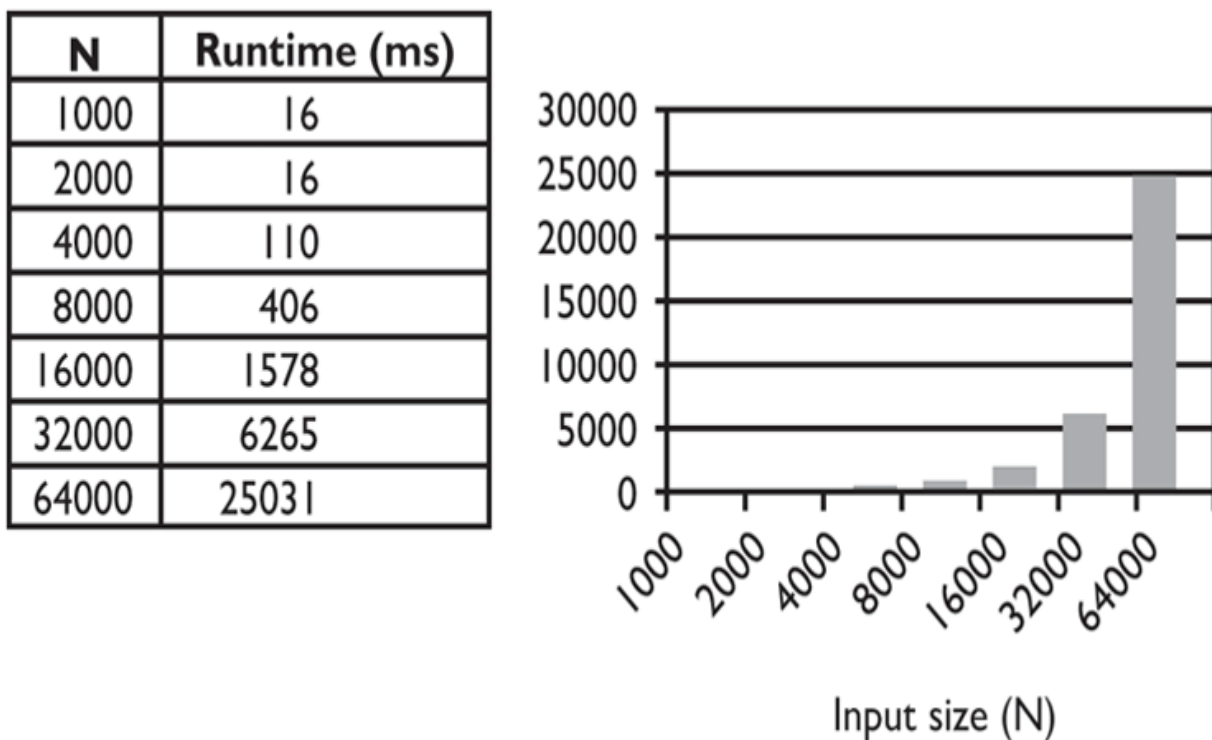
Studying a piece of code and trying to figure out how to speed it up can be deceptively difficult. It's tempting to approach the problem by looking at each line of code and trying to reduce the amount of computation it performs. For example, you may have noticed that our `range` method actually examines every pair of elements in the array twice: For unique integers `i` and `j`, we examine the pair of elements at indexes `(i, j)` as well as the pair at `(j, i)`.

We can perform a minor modification to our `range` method's code by starting each inner `j` loop ahead of `i`, so that we won't examine any pair `(i, j)` where `i ≥ j`. Performing minor modifications like this is sometimes called tweaking an algorithm. The following code implements our tweaked version of the range algorithm:

```
public static int range2(int[] numbers) {  
    int maxDiff = 0;  
    for (int i = 0; i < numbers.length; i++) {  
        for (int j = i + 1; j < numbers.length; j++) {  
            int diff = Math.abs(numbers[j] - numbers[i]);  
            maxDiff = Math.max(maxDiff, diff);  
        }  
    }  
    return maxDiff;  
}
```

Since about half of the possible pairs of `i/j` values are eliminated by this tweak, we'd hope that the code would run about twice as fast. **Figure 13.3** shows its actual measured runtime. As we estimated, the second version is about twice as fast as the first. We could implement other minor tweaks, such as replacing the `Math.max` call with a simple `if` test (which would speed up the algorithm by around 10% more), but there's a more important point to be made. When the input size doubles, the runtime of either version of the algorithm roughly quadruples. Consequently, regardless of which version we use, if the input array is very large the method will be too slow.

Figure 13.3 Runtimes for second version of range algorithm



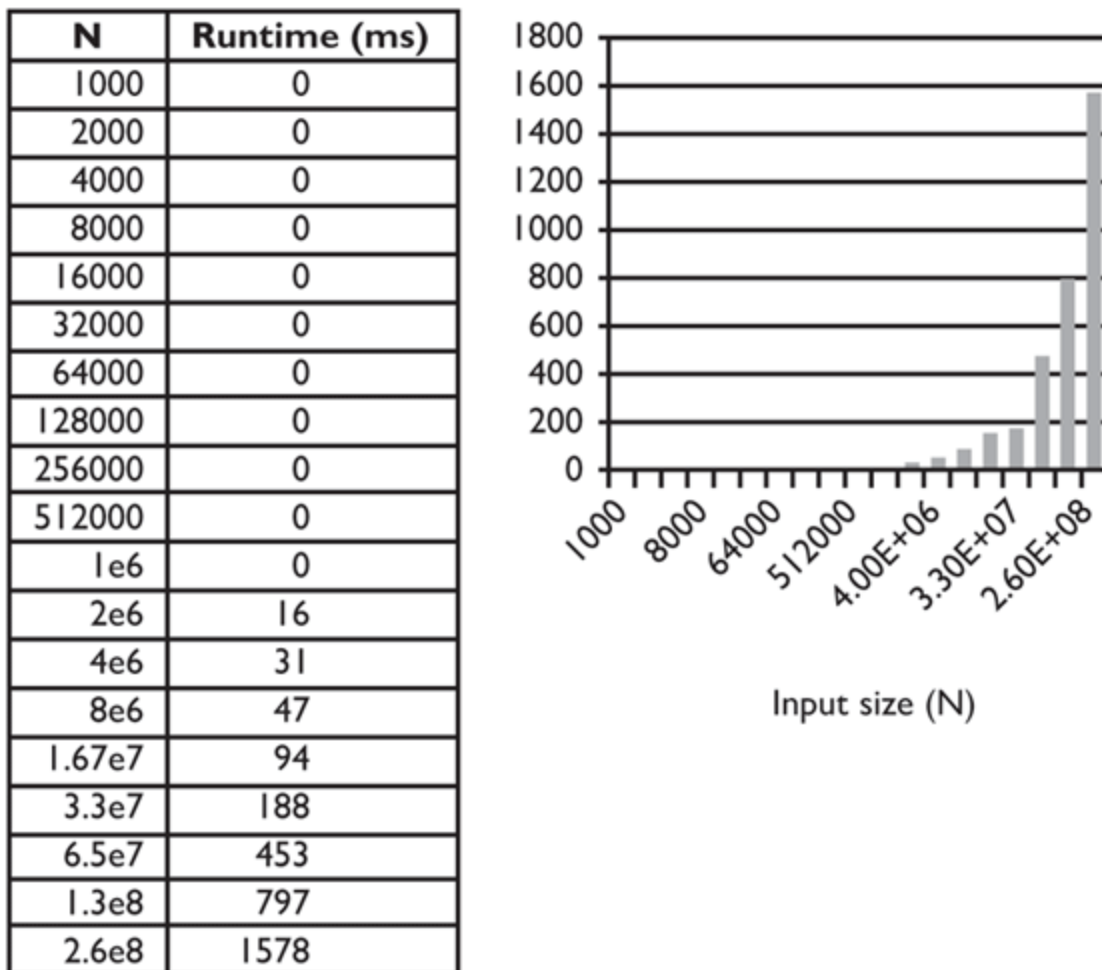
Rather than trying to further tweak our nested loop solution, let's try to think of a more efficient algorithm. As we noted earlier, the range

of values in an array is the difference between the array's largest and smallest elements. We don't really need to examine all pairs of values to find this range; we just need to discover the pair representing the largest and smallest values. We can discover both of these values in a single loop over the array by using a min/max loop, discussed in [Chapter 4](#). The following new algorithm demonstrates this idea:

```
public static int range3(int[] numbers) {  
    int max = numbers[0];  
    int min = max;  
    for (int i = 1; i < numbers.length; i++) {  
        if (numbers[i] > max) {  
            max = numbers[i];  
        } else if (numbers[i] < min) {  
            min = numbers[i];  
        }  
    }  
    return max - min;  
}
```

Since this algorithm passes over the array only once, we'd hope that its runtime would be proportional to the array's length. If the array length doubles, the runtime should double, not quadruple. [Figure 13.4](#) shows its runtime.

Figure 13.4 Runtimes for third version of range algorithm



Our runtime predictions were roughly correct. As the size of the array doubles, the runtime of this new range algorithm approximately doubles as well. The overall runtime of this algorithm is much better; we can examine over a hundred million integers in less than a second.

There are some important observations to take away from this exercise:

- Tweaking an algorithm's code often isn't as powerful an optimization as finding a better algorithm.
- An algorithm's rate of growth, or the amount by which its runtime increases as the input dataset grows, is the standard measure of the complexity of the algorithm.

Complexity Classes

We categorize rates of growth on the basis of their proportion to the input data size N . We call these categories *complexity classes* or *growth rates*.

Complexity Class

A set of algorithms that have a similar relationship between input data size and resource consumption.

The complexity class of a piece of code is determined by looking at the most frequently executed line of code, determining the number of times it is executed, and extracting the highest power of N . For example, if the most frequent line executes $(2N^3 + 4N)$ times, the algorithm is in the “order N^3 ” complexity class, or $O(N^3)$ for

short. The shorthand notation with the capital O is called *big-Oh notation* and is used commonly in algorithm analysis.

Here are some of the most common complexity classes, listed in order from slowest to fastest growth (i.e., from lowest to highest complexity):

- *Constant-time*, or $O(1)$, algorithms have runtimes that don't depend on input size. Some examples of constant-time algorithms would be code to convert Fahrenheit temperatures to Celsius or numerical functions such as `Math.abs`.
- *Logarithmic*, or $O(\log N)$, algorithms typically divide a problem space in half repeatedly until the problem is solved. Binary search is an example of a logarithmic-time algorithm.
- *Linear*, or $O(N)$, algorithms have runtimes that are directly proportional to N (i.e., roughly they double when N doubles). Many algorithms that process each element of a data set are linear, such as algorithms that compute the count, sum, average, maximum, or range of a list of numbers.
- *Log-linear*, or $O(N \log N)$, algorithms typically perform a combination of logarithmic and linear operations, such as executing a logarithmic algorithm over every element of a dataset of size N . Many efficient sorting algorithms, such as merge sort (discussed later in this chapter), are log-linear.

DID YOU KNOW?

Timing Code and the Epoch

Java's method `System.currentTimeMillis` returns the number of milliseconds that have passed since 12:00 AM on January 1, 1970. This method can be used to measure the runtime of an algorithm. Over one trillion milliseconds have passed since the indicated time, so the value is too large to store in a simple `int` value. The milliseconds are instead returned as a value of type `long`, which is a primitive type that's similar to `int` but that is capable of holding much larger values. A `long` can store any number up to 9,223,372,036,854,775,807, or roughly 2^{63} .

The following lines of code show how a piece of code can be timed:

```
long startTime = System.currentTimeMillis();  
<code to be timed>  
long endTime = System.currentTimeMillis();  
System.out.println("Elapsed time (ms): " + (endTime -  
startTime));
```

The choice of January 1, 1970, as the point of reference for system times is an example of an *epoch*, or an instant chosen as the origin of a particular time scale. This particular epoch

was chosen because it matches the epochs of many popular operating systems, including Unix.

For historical reasons, many older Unix operating systems store the time that has passed since the epoch as a 32-bit integer value. However, unspecified problems may occur when this number exceeds its capacity, which is not necessarily a rare event. The clocks of some Unix systems will overflow on January 19, 2038, creating a Year 2038 problem similar to the famous Year 2000 (Y2K) problem.

- *Quadratic*, or $O(N^2)$, algorithms have runtimes that are proportional to the square of the input size. This means that quadratic algorithms' runtimes roughly quadruple when N doubles. The initial versions of the range algorithm developed in the previous section were quadratic algorithms.
- *Cubic*, or $O(N^3)$, algorithms have runtimes that are proportional to the cube of the input size. Such algorithms often make triply nested passes over the input data. Code to multiply two $N \times N$ matrices or to count the number of colinear trios of points in a large `Point` array would be examples of cubic algorithms.
- *Exponential*, or $O(2^N)$, algorithms have runtimes that are proportional to 2 raised to the power of the input size. This means that if the input size increases by just one, the algorithm will take roughly twice as long to execute. One example would be code to print the “power set” of a dataset, which is the set of all possible

subsets of the data. Exponential algorithms are so slow that they should be executed only on very small input datasets.

Table 13.3  presents several hypothetical algorithm runtimes as the input size N grows, assuming that each algorithm requires 100 ms to process 100 elements. Notice that even though they all start at the same runtime for a small input size, as N grows the algorithms in higher complexity classes become so slow that they would be impractical.

Table 13.3 Algorithm Runtime Comparison Chart

When you look at the numbers in **Table 13.3** , you might wonder why anyone bothers to use $O(N^3)$ or $O(2^N)$, algorithms when $O(1)$ and $O(N)$ algorithms are so much faster. The answer is that not all problems can be solved in $O(1)$ or even $O(N)$ time. Computer scientists have been studying classic problems such as searching and sorting for many years, trying to find the most efficient algorithms possible. However, there will likely never be a constant-time algorithm that can sort 1,000,000 elements as quickly as it can sort 10 elements.

For large datasets it's very important to choose the most efficient algorithms possible (i.e., those with the lowest complexity classes). Algorithms with complexity classes of $O(N^2)$ or worse will take a long time to run on extremely large datasets. Keeping this in mind, we'll now examine algorithms to search and sort data.

13.3 Implementing Searching and Sorting Algorithms

In this section, we'll implement methods that search and sort data. We'll start by writing code to search for an integer in an array of integers and return the index where it is found. If the integer doesn't appear in the array, we'll return a negative number. We'll examine two major searching algorithms, sequential and binary search, and discuss the tradeoffs between them.

There are literally hundreds of algorithms to sort data; we'll cover two in detail in this chapter. The first, seen later in this section, is one of the more intuitive algorithms, although it performs poorly on large datasets. The second, examined as a case study in the next section, is one of the fastest general-purpose sorting algorithms that is used in practice today.

Sequential Search

Perhaps the simplest way to search an array is to loop over the elements of the array and check each one to see if it is the target number. As we mentioned earlier, this is called a sequential search because it examines every element in sequence.

We implemented a sequential search of an array of integers in [Chapter 7](#). The code uses a `for` loop and is relatively straightforward. The algorithm returns `-1` if the loop completes without finding the target number:

```
// Sequential search algorithm.  
// Returns the index at which the given target number first  
// appears in the given input array, or -1 if it is not found.  
public static int indexOf(int[] list, int target) {  
    for (int i = 0; i < list.length; i++) {  
        if (list[i] == target) {  
            return i;  
        }  
    }  
    return -1; // not found  
}
```

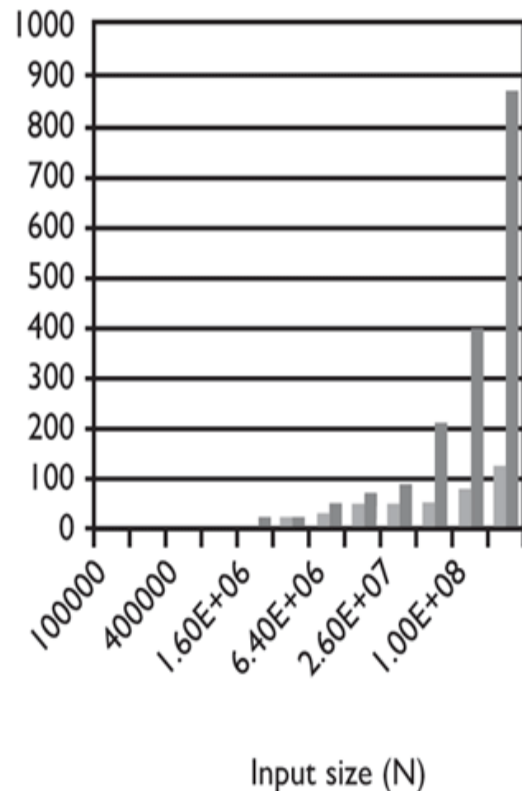
Using the rules stated in the previous section, we predict that the sequential search algorithm is a linear $O(N)$ algorithm because it contains one loop that traverses at most N elements in an array. (We say “at most” because if the algorithm finds the target element, it stops and immediately returns the index.) Next, we’ll time it to verify our prediction.

Figure 13.5 shows actual results of running the sequential search algorithm on randomly generated arrays of integers. Searches were

conducted for a value known to be in the array and for a value not in the array.

Figure 13.5 Runtimes for sequential search algorithm

N	Runtime when element is found (ms)	Runtime when element is not found (ms)
100000	0	0
200000	0	0
400000	0	0
800000	0	0
1.6e6	0	16
3.2e6	15	16
6.4e6	16	31
1.3e7	32	47
2.6e7	37	93
5.1e7	45	203
1.0e8	72	402
2.0e8	125	875



When the algorithm searches for an integer that isn't in the array, it runs somewhat slower, because it can't exit its loop early by finding the target. This scenario raises the question of whether we should judge the algorithm by its fastest or slowest runtime. Often what's most important is the expected behavior for a typical input, or the average of its runtime over all possible inputs. This is called an *average case analysis*. But under certain conditions, we also care

about the fastest possible outcome, the *best case analysis*, and/or the slowest possible outcome, the *worst case analysis*. In this algorithm, the average search looks at approximately half of the array, which is linear or $O(N)$.

Binary Search

Consider a modified version of the searching problem, in which we can assume that the elements of the input array are in sorted order. Does this ordering affect our algorithm? Our existing algorithm will still work correctly, but now we know that we can stop the search if we ever get to a number larger than our target without finding the target. For example, if we're searching an array containing the elements `[1, 4, 5, 7, 7, 9, 10, 12, 56]` for the target value `8`, we can stop searching once we see the `9`.

We might think that such a modification to our sequential search algorithm would significantly speed up the algorithm, but in actuality it doesn't make much difference. The only case in which it speeds up the algorithm noticeably is when it is searching for a relatively small value that isn't found in the array. In fact, when the modified algorithm is searching for a large value that requires the code to examine most or all of the array elements, the algorithm actually performs slower than the original because it has to perform a few more Boolean tests. Most importantly, the algorithm is still $O(N)$, which isn't the optimal solution.

Once again, tweaking the algorithm won't make as much difference as finding another, more efficient algorithm. If the input array is in sorted order, a sequential search isn't the best choice. If you had to instruct a robot how to look up a person's phone number in a phone book, would you tell the robot to read through all the entries on the first page, then the second, and so on until it found the person's name? Not unless you wanted to torture the poor robot. You know that the entries are sorted by name, so you'd tell the robot to flip open the book to somewhere near the middle, then narrow its search toward the first letter of the person's name.

The binary search algorithm discussed previously in this chapter takes advantage of the ordering of the array. A binary search keeps track of the range of the array that is currently of interest. (Initially, this range is the whole array.) The algorithm repeatedly examines the center element of the array and uses its value to eliminate half of the range of interest. If the center element is smaller than the target, the lower half of the range is eliminated; if the center element is larger than the target, the upper half is eliminated.

As the algorithm runs, we must keep track of three indexes:

- The minimum index of interest (`min`)
- The maximum index of interest (`max`)
- The middle index, halfway between the minimum and maximum, which will be examined during each pass of the algorithm (`mid`)

The algorithm repeatedly examines the element at the middle index and uses it to trim the range of indexes of interest in half. If we examine the middle element and find it's too small, we will eliminate all elements between `min` and `mid` from consideration. If the middle element is too large, we will eliminate all elements between `mid` and `max` from consideration.

Consider the following array:

```
int[] numbers = {11, 18, 29, 37, 42, 49, 51, 63,
                 69, 72, 77, 82, 88, 91, 98};
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
11	18	29	37	42	49	51	63	69	72	77	82	88	91	98

Let's run a binary search on the array for a target value of `77`. We'll start at the middle element, which is at index $14/2$, or 7. The following diagrams show the `min`, `mid`, and `max` at each step of the algorithm:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
11	18	29	37	42	49	51	63	69	72	77	82	88	91	98
↑ min							↑ mid							↑ max

```
observation: numbers[7] < 77
```

decision: search indexes 8 – 14

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
11	18	29	37	42	49	51	63	69	72	77	82	88	91	98
								↑ min			↑ mid			↑ max

```
observation: numbers[11] > 77
```

decision: search indexes 8 – 10

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
11	18	29	37	42	49	51	63	69	72	77	82	88	91	98

↑ ↑ ↑
min mid max

```
observation: numbers[9] < 77
```

decision: search index 10

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
11	18	29	37	42	49	51	63	69	72	77	82	88	91	98

↑
min
mid
max

observation: `numbers[10] == 77`

decision: `return 10`

What about when we're searching for an element that isn't found in the array? Let's say we're searching for the value `78` instead of `77`. The steps of the algorithm will be the same, except that on the fourth pass the algorithm will reach `77` instead of the desired value, `78`. The algorithm will have eliminated the entire range without finding the target and will know that it should stop. Another way to describe the process is that the algorithm loops until the `min` and `max` have crossed each other.

The following code implements the binary search algorithm. Its loop repeats until the target is found or until the `min` and `max` have crossed:

```
// Binary search algorithm.
```

```

// Returns an index at which the target
// appears in the given input array, or -1 if not found.
// pre: array is sorted.
public static int binarySearch(int[] numbers, int target) {
    int min = 0;
    int max = numbers.length - 1;
    while (min <= max) {
        int mid = (max + min) / 2;
        if (numbers[mid] == target) {
            return mid;    // found it!
        } else if (numbers[mid] < target) {
            min = mid + 1;  // too small
        } else { // numbers[mid] > target
            max = mid - 1;  // too large
        }
    }
    return -1;    // not found
}

```

We won't bother to show a runtime chart for the binary search algorithm, because there would be nothing to draw on the chart. This algorithm is so fast that the computer's clock has trouble measuring its runtime. On a modern computer, even an array of over 100,000,000 elements registers as taking 0 ms to search!

While this is an impressive result, it makes it harder for us to empirically examine the runtime. What is the complexity class of the binary search algorithm? The fact that it finishes so quickly tempts us

to conclude that it's a constant-time, or $O(1)$, algorithm. But it doesn't seem right that a method with a loop in it would take a constant amount of time to execute. There is a relation between the runtime and the input size, because the larger the input is, the more times we must divide our `min-max` range in half to arrive at a single element.

We could say that 2 raised to the number of repetitions is approximately equal to the input size N :

$$2^{\text{repetitions}} \cong N$$

Using some algebra and taking a logarithm base-2 of both sides of the equation, we find that

$$\text{repetitions} \cong \log_2 N$$

We conclude that the binary search algorithm is in the logarithmic complexity class, or $O(\log N)$.

The runtime of the binary search algorithm doesn't differ much between the best and worst cases. In the best case, the algorithm finds its target value in the middle on the first check. In the worst case, the code must perform the full $(\log N)$ comparisons. But since logarithms are small numbers ($\log_2 1,000,000$ is roughly 20), the performance is still excellent in the worst case.

Recursive Binary Search

In the previous section, binary search was implemented using an iterative algorithm with a `while` loop. But the algorithm can also be implemented elegantly using the concept of recursion introduced in [Chapter 12](#). The recursive version of the method should accept the same parameters as the standard binary search:

```
public static int binarySearchR(int[] numbers, int target) {  
    ...  
}
```

But this header will not make it easy to write a recursive solution to the problem. Recall that the essence of a recursive solution is to break down the problem into smaller pieces and then solve the sub-problem(s). In this algorithm, the way to shrink the problem is to examine a smaller and smaller portion of the array until we find the right index. To do this, we can change our method to accept additional parameters for the range of indexes (min and max) that currently are being examined. Rather than changing the header just shown, we can add a private recursive helper with the additional parameters as described in [Chapter 12](#). The public method can start the recursive process by calling the helper with `0` and `length - 1` as its minimum and maximum indexes to examine, respectively:

```
// Recursive binary search algorithm.
```

```

// Returns an index at which the target
// appears in the given input array, or -1 if not found.
// pre: array is sorted.

public static int binarySearchR(int[] numbers, int target) {
    return binarySearchR(numbers, target, 0, numbers.length -
1);
}

// private recursive helper to implement binary search
private static int binarySearchR(int[] numbers, int target,
int min, int max) {
    ...
}

```

Our recursive binary search method accepts the minimum and maximum indexes of interest as parameters. On each pass of the recursive algorithm, the code examines the middle element. If this element is too small, the code recursively examines only the right half of the array. If the middle element is too large, the code recursively examines only the left half. This process repeats, recursively calling itself with different values of `min` and `max`, until it finds the target or until the entire array has been eliminated from consideration. The following code implements the algorithm:

```

// Recursive binary search algorithm.
// Returns an index at which the target

```

```

// appears in the given input array, or -1 if not found.
// pre: array is sorted.
private static int binarySearchR(int[] numbers, int target,
                                int min, int max) {
    // base case
    if (min > max) {
        return -1; // not found
    } else {
        // recursive case
        int mid = (max + min) / 2;
        if (numbers[mid] == target) {
            return mid;
        } else if (numbers[mid] < target) {
            return binarySearchR(numbers, target, mid + 1,
max);
        } else {
            return binarySearchR(numbers, target, min, mid -
1);
        }
    }
}

```

Some instructors don't like recursive versions of methods like binary search because there is a nonrecursive solution that's fairly easy to write and because recursion tends to have poor runtime performance because of the extra method calls it generates. However, that doesn't pose a problem here. The runtime of the recursive version of our binary search method is still $O(\log N)$,

because it's essentially performing the same computation; it's still cutting the input in half at each step. In fact, the recursive version is fast enough that the computer still can't time it accurately. It produces a runtime of 0 ms even on arrays of tens of millions of integers.

In general, analyzing the runtimes of recursive algorithms is tricky. Recursive runtime analysis often requires a technique called *recurrence relations*, which are mathematical relations that describe an algorithm's runtime in terms of itself. That's a complex topic for a later course that won't be covered in this textbook.

DID YOU KNOW?

Binary Search Details

There are a few interesting things about Java's implementation of binary search in `Arrays.binarySearch` and `Collections.binarySearch` that we haven't mentioned yet. Take a look at this text from the Javadoc documentation of the `binarySearch` method:

The array **must** be sorted (as by the `sort` method, above) prior to making this call. If it is not sorted, the results are undefined. If the array contains multiple elements with the specified value, there is no guarantee which one will be found.

Binary search depends on the array being sorted. If it isn't sorted, the documentation says the results are undefined. What does that mean? Why doesn't the algorithm just sort the array for you if it's unsorted?

There are two problems with that idea, both essentially related to runtime performance. For one, sorting takes much longer ($O(N \log N)$ time) than a binary search ($O(\log N)$ time). Second, to even discover that you need to sort the array, you'd need to look at each element to determine whether they're in order, which takes $O(N)$ time. Essentially, the cost of examining the array and sorting it if necessary would be too great.

Even if the cost of sorting the array weren't so large, the client of the `binarySearch` method probably won't want its array modified by the `binarySearch` method. Searching is supposed to be a read-only operation, not one that rearranges the array.

Let's look at the other part of the previous quote: "If the array contains multiple elements with the specified value, there is no guarantee which one will be found." We didn't mention this earlier when we were discussing the algorithm, but in the case of duplicates, binary search isn't guaranteed to find the first occurrence of the element, because the moment it finds an occurrence it stops.

Here's another interesting blurb from the `binarySearch` documentation:

Returns: index of the search key, if it is contained in the list; otherwise, `(- (insertion point) - 1)`. The insertion point is defined as the point at which the key would be inserted into the list: the index of the first element greater than the key, or `list.size()`, if all elements in the list are less than the specified key. Note that this guarantees that the return value will be `>= 0` if and only if the key is found.

Rather than returning `-1` for an unsuccessful search, the `Arrays.binarySearch` and `Collections.binarySearch` methods return `(-index - 1)`, where `index` is the last index the algorithm examined before giving up, which is also the index of the first element whose value is greater than the target. The documentation for these methods calls this value the *insertion point* because if you wanted to add the target to the array in sorted position, you'd place it at that index. For example, because a binary search of the following array for the value `45` would finish at index 5, the algorithm would return `-6`:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
11	18	29	37	42	49	51	63	69	72	77	82	88	91	98

If you were maintaining this sorted array and wanted to add `45` to it at the proper index to retain the sorted order, you could call `Arrays.binarySearch`, get the result of `-6`, negate it, and subtract 1 from it to get the index at which you should insert the value. This is much faster than linearly searching

for the place to add the value or adding the value at the end and resorting the array.

We could modify our own binary search code to match this behavior by changing the last line of the method's body to the following line:

```
return -min - 1; // not found
```

Searching Objects

Searching for a particular object in an array of objects requires a few modifications to our searching code from the previous sections. Let's look at a sequential object search first, because it will work with any type of object. The most important modification to make to the code is to ensure that it uses the `equals` method to compare objects for equality:

```
// Sequential search algorithm.  
// Returns the index at which the target first  
// appears in the given input array, or -1 if not found.  
public static int indexOf(Object[] objects, Object target) {
```

```

    for (int i = 0; i < objects.length; i++) {
        if (objects[i].equals(target)) {
            return i; // found it!
        }
    }

    return -1; // not found
}

```

If we want to do a binary search on objects, the objects must have an ordering (in other words, must be of a type that implements the `Comparable` interface or must be provided with a comparator) and the elements in the array or collection must be in sorted order. One common example would be an array of `Strings`. Since we can't use relational operators like `<` and `>=` on objects, we must call the `compareTo` method on pairs of `String` objects and examine the value it returns. The following code implements a binary search on an array of `String`s:

```

// Binary search algorithm that works with Strings.
// Returns an index at which the given target String
// appears in the given input array, or -1 if not found.
// pre: array is sorted
public static int binarySearch(String[] strings, String target)
{
    int min = 0;

```

```
int max = strings.length - 1;
while (min <= max) {
    int mid = (max + min) / 2;
    int compare = strings[mid].compareTo(target);
    if (compare == 0) {
        return mid;    // found it!
    } else if (compare < 0) {
        min = mid + 1; // too small
    } else { // compare > 0
        max = mid - 1; // too large
    }
}
return -1;    // not found
}
```

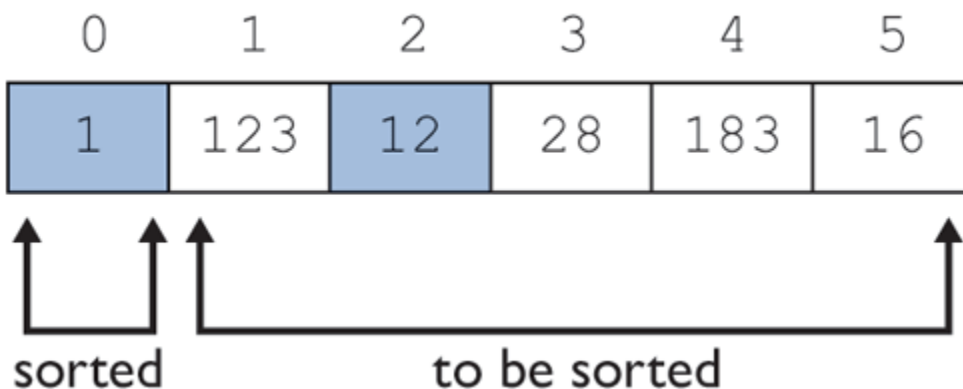
Selection Sort

Selection sort is a well-known sorting algorithm that makes many passes over an input array to put its elements into sorted order. Each time it runs through a loop, it selects the smallest value and puts it in the proper place near the front of the array. Consider the following array:

```
int[] nums = {12, 123, 1, 28, 183, 16};
```

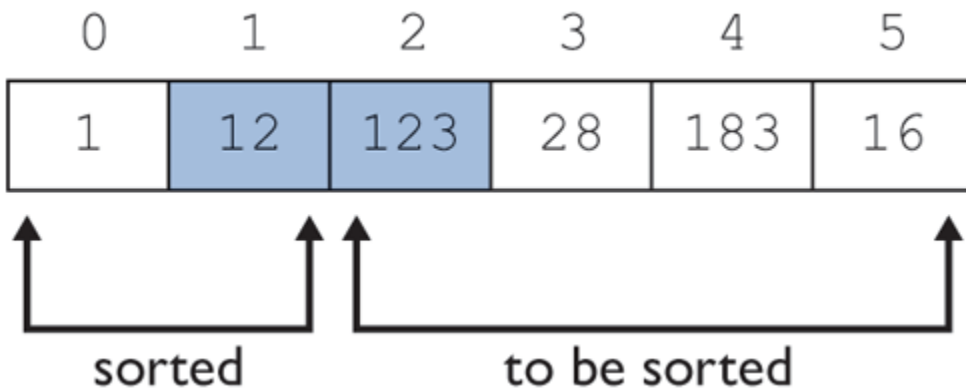
0	1	2	3	4	5
12	123	1	28	183	16

How would you put its elements into order from smallest to largest? The selection sort algorithm conceptually divides the array into two pieces: sorted elements at the front and unsorted elements at the end. The first step of the selection sort makes a pass over the array and finds the smallest number. In the sample array, the smallest is `nums[2]`, which equals `1`. The algorithm then swaps the smallest value with the value in the first position in the array, so that the smallest value will be at the front of the array. In this case, `nums[0]` and `nums[2]` are swapped:

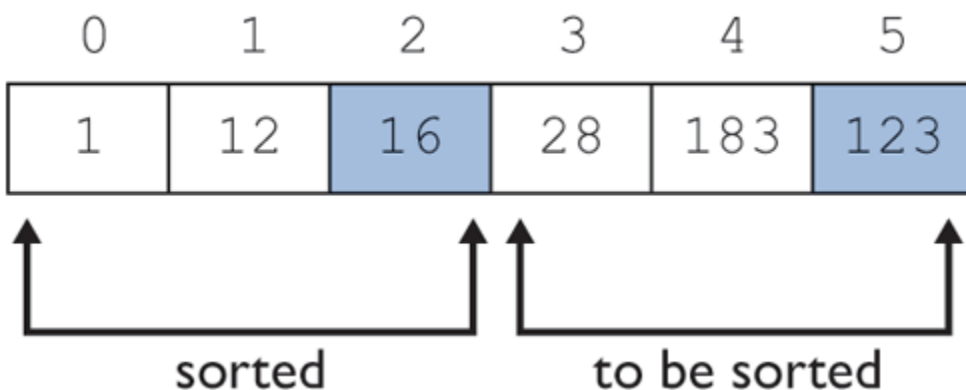


The element at index 0 now has the right value, and only the elements at indexes 1 through 5 remain to be ordered. The algorithm repeats the process of scanning the unsorted portion of the array and looking for the smallest element. On the second pass, it scans the remaining five elements and finds that `nums[2]`, which equals `12`,

is the smallest element. The program swaps this value with `nums[1]`. After this swap, the sorted area of the array consists of its first two indexes:



Now `nums[0]` and `nums[1]` have the correct values. The third pass of the algorithm scans the remaining four unsorted elements and finds that the smallest one is `nums[5]`, which equals 16. It swaps this element with `nums[2]`.



The algorithm continues this process until all the elements have the proper values. Each pass involves a scan followed by a swap. The scan/swap occurs five times to process six elements. You don't need

to perform a sixth scan/swap because, if the first five elements have the correct values, the sixth element will be correct as well.

Here is a pseudocode description of the execution of the selection sort algorithm over an array `nums` that has six elements:

```
for (each i from 0 to 4) {  
    scan nums[i] through nums[5] for the smallest value.  
    swap nums[i] with the smallest element found in the scan.  
}
```

You can write pseudocode like the following for the scan:

```
smallest = lowest array index of interest.  
for (all other index values of interest) {  
    if (nums[index] < nums[smallest]) {  
        smallest = index.  
    }  
}
```

You can then incorporate this pseudocode into your larger pseudocode as follows:

```

for (each i from 0 to 4) {
    smallest = i.
    for (each index between (i + 1) and 5) {
        if (nums[index] < nums[smallest]) {
            smallest = index.
        }
    }
    swap nums[i] with nums[smallest].
}

```

You can translate this pseudocode almost directly into Java, except for the swap process. In [Chapter 7](#), we wrote a `swap` method to swap two elements of an array. We can reuse it here:

```

public static void swap(int[] list, int i, int j) {
    int temp = list[i];
    list[i] = list[j];
    list[j] = temp;
}

```

We can also modify the code to work with arrays of any size. The following code implements the overall selection sort algorithm:

```

// places the elements of the given array into sorted order
// using the selection sort algorithm
// post: array is in sorted (nondecreasing) order
public static void selectionSort(int[] a) {
    for (int i = 0; i < a.length - 1; i++) {
        // find index of smallest element
        int smallest = i;
        for (int j = i + 1; j < a.length; j++) {
            if (a[j] < a[smallest]) {
                smallest = j;
            }
        }
        swap(a, i, smallest); // swap smallest to front
    }
}

```

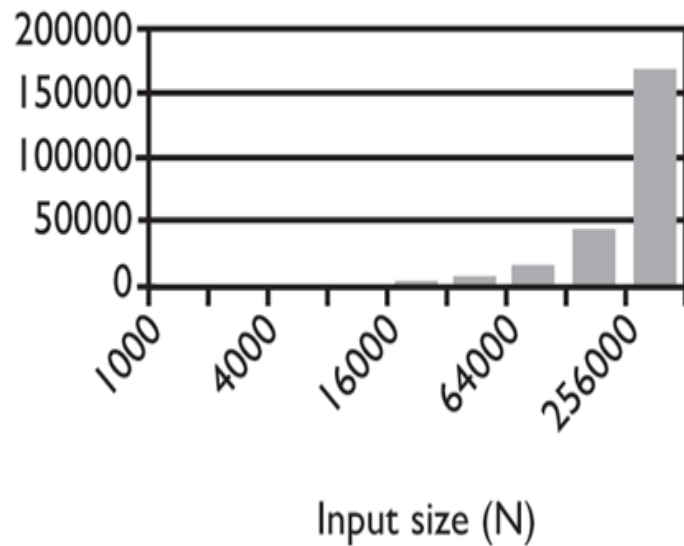
Since selection sort makes roughly N passes over an array of N elements, its performance is $O(N^2)$. Technically, it examines $N + (N - 1) + (N - 2) + \dots + 3 + 2 + 1$ elements, because each pass starts one index ahead of where the last one started.

Chapter 3  mentioned a mathematical identity which states that the sum of all integers from 1 to any maximum value N equals $(N)(N + 1)/2$, which is just over $\frac{1}{2}N^2$. **Figure 13.6**  supports this analysis, because the runtime quadruples every time the input size N is doubled, which is characteristic of an N^2 algorithm. The

algorithm becomes impractically slow once the number of elements reaches tens of thousands.

Figure 13.6 Runtimes for selection sort algorithm

N	Runtime (ms)
1000	0
2000	16
4000	47
8000	234
16000	657
32000	2562
64000	10265
128000	41141
256000	164985



The current `selectionSort` code will sort arrays of integer values, but you could adapt it to sort `Comparable` objects such as `Strings` using the techniques covered in the previous section on searching objects. For example, you could use the `compareTo` and `equals` methods to compare objects rather than relational operators like `<` and `>=`.

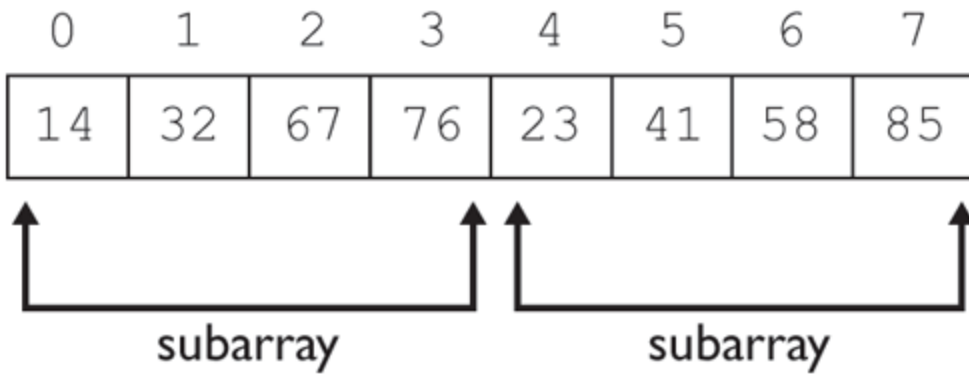
13.4 Case Study: Implementing Merge Sort

There are other algorithms similar to selection sort that make many passes over the array and swap various elements on each pass. An algorithm that searches for inverted pairs of elements and swaps them into order in this way cannot run faster than $O(N^2)$, on average. However, there is a better algorithm that breaks this barrier.

The *merge sort* algorithm is named for the observation that if you have two sorted subarrays, you can easily merge them into a single sorted array. For example, consider the following array:

```
int[] list = {14, 32, 67, 76, 23, 41, 58, 85};
```

You can think of it as two subarray halves, each of which (because of the element values we chose) happens to be sorted:



The following pseudocode provides the basic idea of the merge sort algorithm:

```
split the array into two halves.  
sort the left half.  
sort the right half.  
merge the two halves.
```

Let's look at splitting the array and merging halves first; then we'll talk about the sorting.

Splitting and Merging Arrays

Splitting one array into its two halves is relatively straightforward. We'll set a midpoint at one half of the length of the array and consider everything before this midpoint to be part of the "left" half and everything that follows it to be in the "right" half. We can use the

method `Arrays.copyOfRange` to extract the halves of an array as new arrays. The “left” half is from range 0 to half the length, and the “right” half is from half the length to the full length:

```
// split array into two halves  
int[] left = Arrays.copyOfRange(a, 0, a.length / 2);  
int[] right = Arrays.copyOfRange(a, a.length / 2, a.length);
```

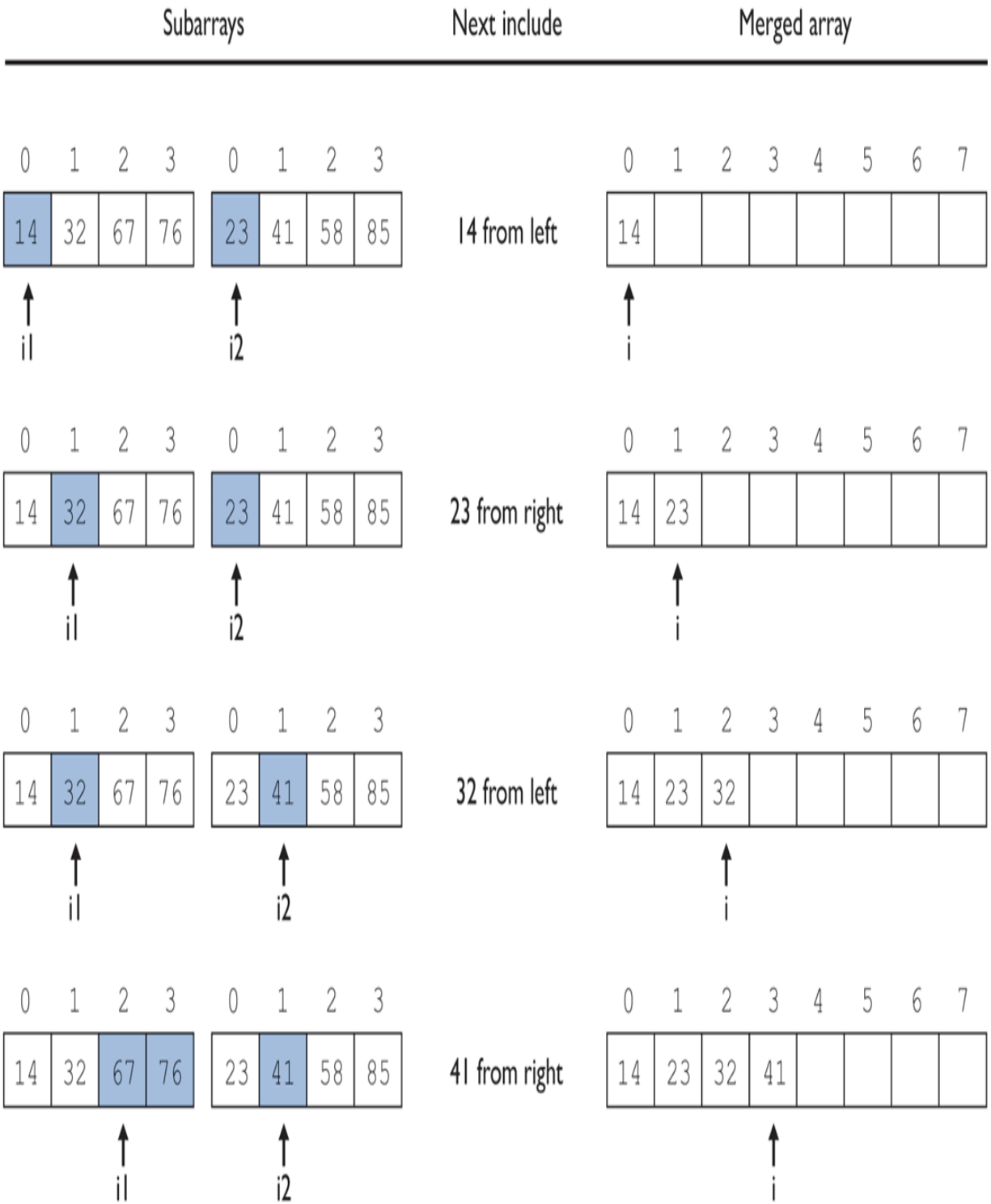
We will need to sort these left/right halves, then merge them into a sorted whole. For now, let’s think about how we would merge two sorted subarrays. (We’ll come back to sorting them later.) Suppose that you have two stacks of exam papers, each sorted alphabetically by name, and you need to combine them into a single stack sorted by name. The simplest algorithm is to place both stacks in front of you, look at the top paper of each stack, pick up the paper that comes first in alphabetical order, and put it face down into a third pile. You then repeat this process, comparing the papers on the top of each stack and placing the one that comes first face down on the merged stack, until one of your two original stacks is empty. Once one is empty, you just grab the entire remaining stack and place it on your merged pile.

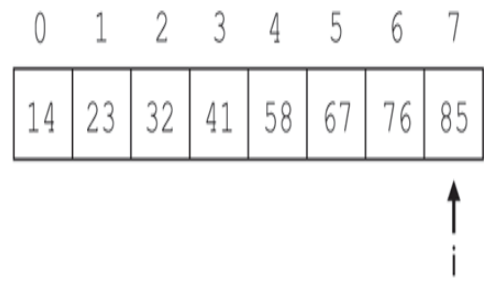
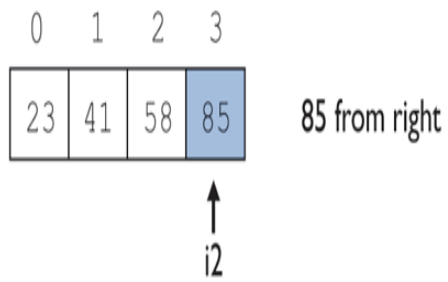
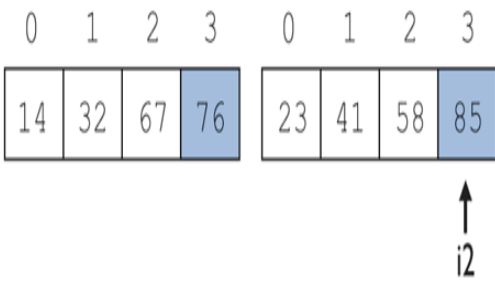
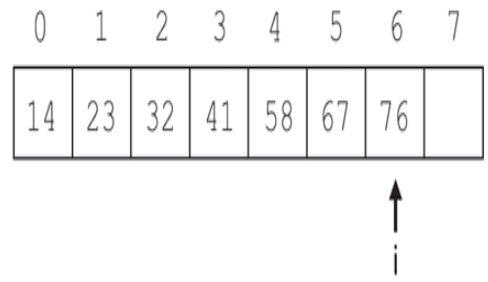
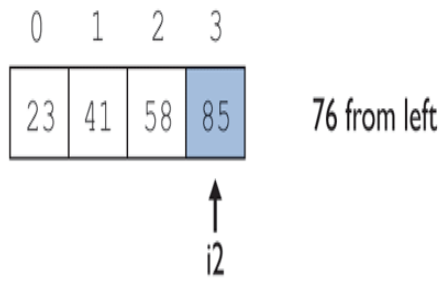
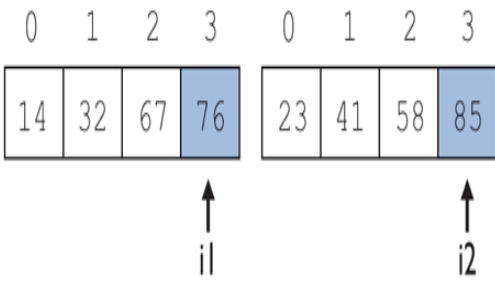
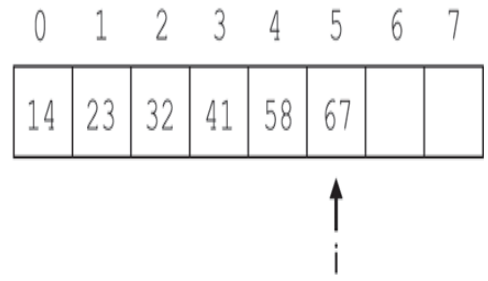
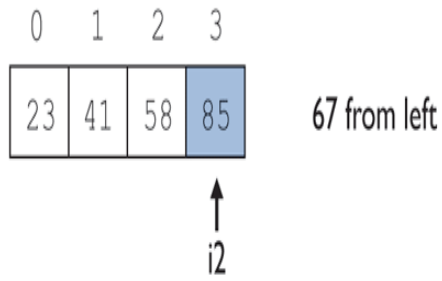
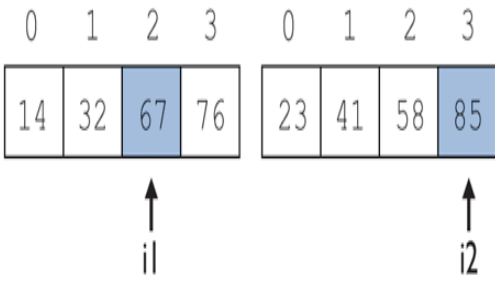
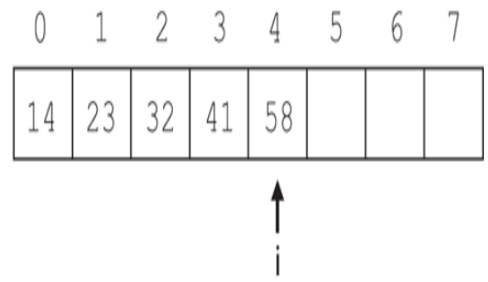
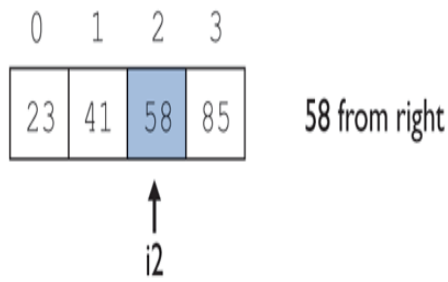
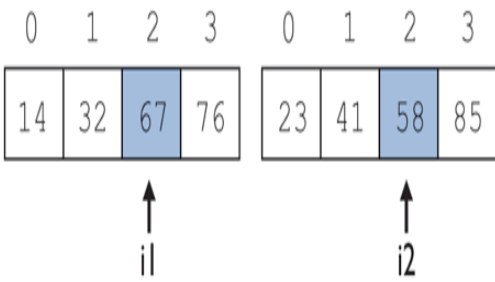
The idea behind merging two sorted arrays is similar, except that instead of physically removing papers (integers) from the piles (subarrays), we’ll keep an index for each subarray and increment

that index as we process a given element. Here is a pseudocode description of the merging algorithm:

```
i1 = 0.    // left index
i2 = 0.    // right index
for (number of elements in entire array) {
    if (left value at i1 <= right value at i2) {
        include value from left array in new array.
        i1++;
    } else {
        include value from right array in new array.
        i2++;
    }
}
```

Here is a trace of the eight steps to merge the two subarrays into a sorted array:





The following code is an initial attempt to implement the merge algorithm that was just described:

```
// initial incorrect attempt

public static void merge(int[] result, int[] left, int[] right)
{
    int i1 = 0;    // index into left array
    int i2 = 0;    // index into right array
    for (int i = 0; i < result.length; i++) {
        if (left[i1] <= right[i2]) {
            result[i] = left[i1];    // take from left
            i1++;
        } else {
            result[i] = right[i2];    // take from right
            i2++;
        }
    }
}
}
```

The preceding code is incorrect and will cause an out-of-bounds exception. After the program completes the seventh step of the preceding diagram, all of the elements in the left subarray will have been consumed and the left index `i1` will run off the end of the subarray. Then, when the code tries to access element `left[i1]`, it will crash. A similar problem would occur if the right index `i2` exceeded the bounds of the right array.

We need to modify our code to remain within the bounds of the arrays. The `if/else` logic needs to ensure that the index `i1` or `i2` is

within the array bounds before the program attempts to access the appropriate element. The simple test in the pseudocode needs to be expanded:

```
if (i2 has passed the end of the right array, or
    left element at i1 <= right element at i2) {
    take from left.
} else {
    take from right.
}
```

The following second version of the code correctly implements the merging behavior. The preconditions and postconditions of the method are documented in comments:

```
// Merges the given left and right arrays into the given
// result array. Second, working version.
// pre : result is empty; left/right are sorted
// post: result contains result of merging sorted lists.
public static void merge(int[] result, int[] left, int[] right)
{
    int i1 = 0;    // index into left array
    int i2 = 0;    // index into right array
    for (int i = 0; i < result.length; i++) {
```

```

        if (i2 >= right.length || (i1 < left.length &&
                                   left[i1] <= right[i2])) {
            result[i] = left[i1];    // take from left
            i1++;
        } else {
            result[i] = right[i2];   // take from right
            i2++;
        }
    }
}

```

Recursive Merge Sort

We've written the code to split an array into halves and to merge the sorted halves into a sorted whole. The overall merge sort method now looks like this:

```

public static void mergeSort(int[] a) {
    // split array into two halves
    int[] left = Arrays.copyOfRange(a, 0, a.length / 2);
    int[] right = Arrays.copyOfRange(a, a.length / 2,
a.length);

    // sort the two halves

```

```
...  
    // merge the sorted halves into a sorted whole  
    merge(a, left, right);  
}
```

The last piece of our program is the code to sort each half of the array. How can we sort the halves? We could call the `selectionSort` method created earlier in this chapter on the two halves. But in the previous chapter we discussed the recursive “leap of faith,” the belief that our own method will work properly to solve a smaller version of the same problem. In this case, a better approach is to merge sort the two smaller halves. We can recursively call our own `mergeSort` method on the array halves, and if it’s written correctly, it’ll put each of them into sorted order. Our original pseudocode can now be rewritten as the following pseudocode:

```
split the array into two halves.  
merge sort the left half.  
merge sort the right half.  
merge the two halves.
```

If we’re making our merge sort algorithm recursive, it needs to have a base case and a recursive case. The preceding pseudocode specifies the recursive case, but for the base case, what are the simplest arrays to sort? An array with either no elements or just one

element doesn't need to be sorted at all. At least two elements must be present in order for them to appear in the wrong order, so the simple case would be an array with a length less than 2. This means that our final pseudocode for the merge sort method is the following:

```
if (array length is more than 1) {  
    split the array into two halves.  
    merge sort the left half.  
    merge sort the right half.  
    merge the two halves.  
}
```

No `else` case is needed because if the array size is 0 or 1, we don't need to do anything to the array. This recursive algorithm has an empty base case.

The following method implements the complete merge sort algorithm:

```
// Places the elements of the given array into sorted order  
// using the merge sort algorithm.  
// post: array is in sorted (nondecreasing) order  
public static void mergeSort(int[] a) {  
    if (a.length > 1) {
```

```

        // split array into two halves
        int[] left = Arrays.copyOfRange(a, 0, a.length / 2);
        int[] right = Arrays.copyOfRange(a, a.length / 2,
a.length);

        // recursively sort the two halves
        mergeSort(left);
        mergeSort(right);

        // merge the sorted halves into a sorted whole
        merge(a, left, right);
    }
}

```

To get a better idea of the algorithm in action, we'll temporarily insert a few `println` statements into its code and run the method on the eight-element sample array shown previously in this section. We'll insert the following `println` statement at the start of the `mergeSort` method:

```

// at start of mergeSort method
System.out.println("sorting " + Arrays.toString(a));

```

We'll also put the following `println` statement at the start of the `merge` method:

```
// at start of merge method  
System.out.println("merging " + Arrays.toString(left) +  
                    " and " + Arrays.toString(right));
```

Here is the output from running `mergeSort` on the example array:

```
sorting [14, 32, 67, 76, 23, 41, 58, 85]  
sorting [14, 32, 67, 76]  
sorting [14, 32]  
sorting [14]  
sorting [32]  
merging [14] and [32]  
sorting [67, 76]  
sorting [67]  
sorting [76]  
merging [67] and [76]  
merging [14, 32] and [67, 76]  
sorting [23, 41, 58, 85]  
sorting [23, 41]  
sorting [23]  
sorting [41]  
merging [23] and [41]  
sorting [58, 85]  
sorting [58]
```

```
sorting [85]
merging [58] and [85]
merging [23, 41] and [58, 85]
merging [14, 32, 67, 76] and [23, 41, 58, 85]
```

It's also important to test the code on an array that doesn't divide into subarrays of exactly equal size (i.e., one whose overall length is not a power of 2). Because it employs integer division, our code makes the left subarray one element smaller than the right subarray when the size is odd. Given an initial five-element list of [14, 32, 67, 76, 23] the algorithm prints the following output:

```
sorting [14, 32, 67, 76, 23]
sorting [14, 32]
sorting [14]
sorting [32]
merging [14] and [32]
sorting [67, 76, 23]
sorting [67]
sorting [76, 23]
sorting [76]
sorting [23]
merging [76] and [23]
merging [67] and [23, 76]
merging [14, 32] and [23, 67, 76]
```

Complete Program

The complete program containing the merge sort code follows. Its `main` method constructs a sample array and sorts it using the algorithm:

```
1 // This program implements merge sort for arrays of
  integers.
2 import java.util.*;
3
4 public class MergeSort {
5     public static void main(String[] args) {
6         int[] list = {14, 32, 67, 76, 23, 41, 58, 85};
7         System.out.println("before: " +
  Arrays.toString(list));
8         mergeSort(list);
9         System.out.println("after : " +
  Arrays.toString(list));
10    }
11
12    // Places the elements of the given array into sorted
  order
13    // using the merge sort algorithm.
14    // post: array is in sorted (nondecreasing) order
15    public static void mergeSort(int[] a) {
```

```

16         if (a.length > 1) {
17             // split array into two halves
18             int[] left = Arrays.copyOfRange(a, 0, a.length /
19 );
20             int[] right = Arrays.copyOfRange(a, a.length /
21 2,
22             a.length);
23             // recursively sort the two halves
24             mergeSort(left);
25             mergeSort(right);
26             // merge the sorted halves into a sorted whole
27             merge(a, left, right);
28         }
29     }
30
31     // Merges the given left and right arrays into the
32     // result array.
33     // pre : result is empty; left/right are sorted
34     // post: result contains result of merging sorted
35     // lists;
36     public static void merge(int[] result, int[] left, int[]
37 right) {
38         int i1 = 0;    // index into left array
39         int i2 = 0;    // index into right array
40         for (int i = 0; i < result.length; i++) {

```

```

39         if (i2 >= right.length || (i1 < left.length &&
40             left[i1] <=
right[i2])) {
41             result[i] = left[i1];    // take from left
42             i1++;
43         } else {
44             result[i] = right[i2];    // take from
right
45             i2++;
46         }
47     }
48 }
49 }

```

The program produces the following output:

```

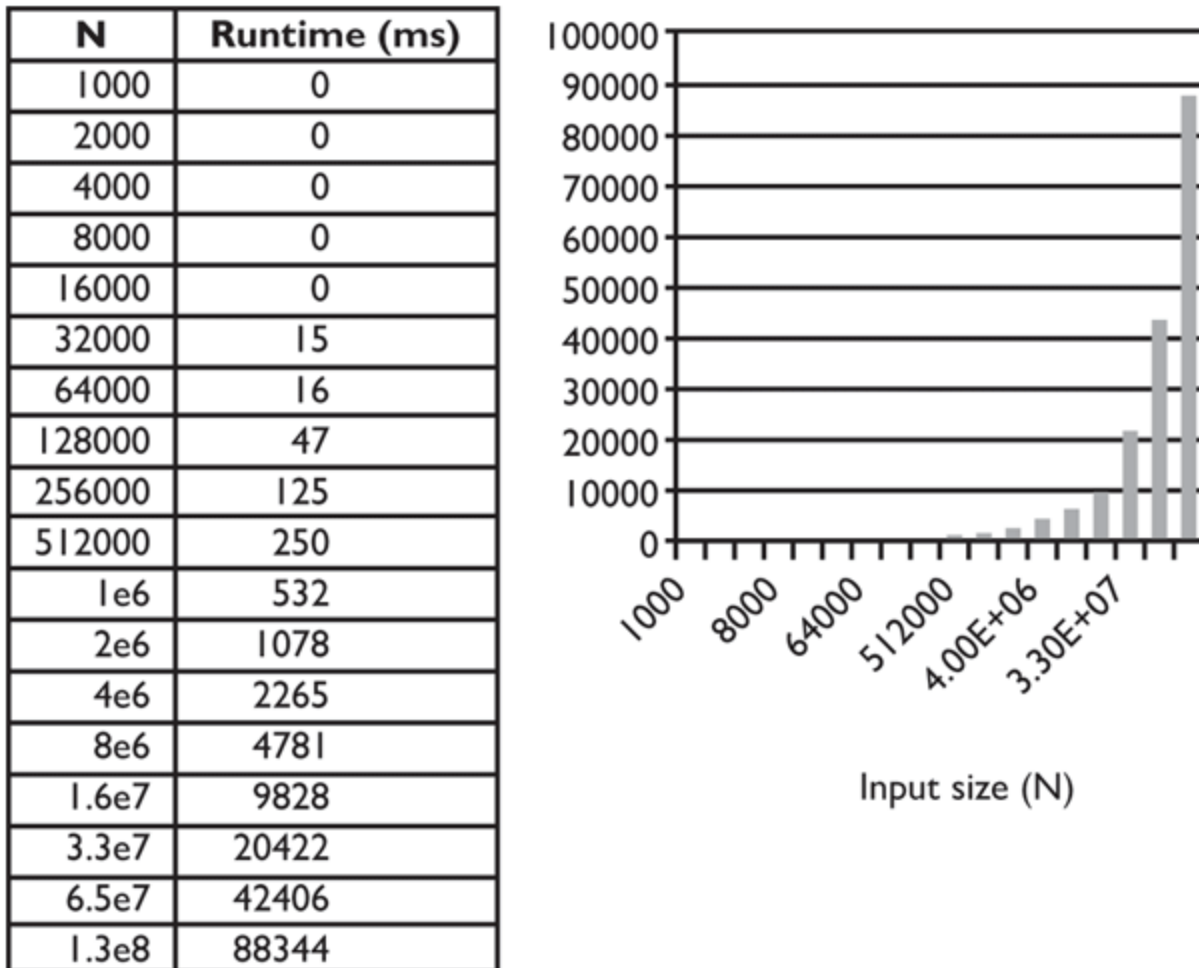
before: [14, 32, 67, 76, 23, 41, 58, 85]
after:  [14, 23, 32, 41, 58, 67, 76, 85]

```

Figure 13.7  demonstrates the performance of our merge sort algorithm on a modern computer. The merge sort algorithm's performance is much better than that of the selection sort algorithm. For example, whereas our selection sort test run needed over 41 seconds to sort 128,000 elements, the merge sort algorithm handled the same job in a blistering 47 milliseconds. But what is merge sort's complexity class? It looks almost like an $O(N)$ algorithm, because

the runtime only slightly more than doubles when we double the array size.

Figure 13.7 Runtimes for merge sort algorithm



However, merge sort is actually an $O(N \log N)$ algorithm. A formal proof of this statement is beyond the scope of this book, but a common-sense chain of reasoning is as follows: We have to split the array in half repeatedly until we hit the algorithm's base case, in which the subarrays each contain 1 element. For an array of size N , we must split the array $\log_2 N$ times. At each of those $\log N$ steps,

we have to do a linear operation of order N (merging the halves after they're sorted). Multiplying these operations' runtimes together produces a $O(N \log N)$ overall runtime.

The preceding algorithm runtime analysis is informal and not rigorous. As with other recursive algorithms, a precise analysis of merge sort's performance is complicated and requires mathematical techniques such as recurrence relations, which are not discussed in this book.

DID YOU KNOW?

Cost of Java Array Allocation

Earlier in the chapter, we mentioned that variable declaration and assignment can be thought of as taking some constant amount of time. That is usually true, but there is one important exception: initializing arrays. Suppose you were to execute this line of code:

```
int[] list = new int[n];
```

Executing this line of code requires the computer to allocate a block of memory for the array. That can generally be done in a constant amount of time. But Java also insists on auto-

initializing all array elements to 0 and that takes time proportional to n .

This subtlety of Java has stung several textbook authors who were translating textbooks from C and C++ into Java. When they were writing a solution to the merge sort algorithm, they included code that would allocate a temporary array as large as the entire array every time they did a merging operation.

For an array of size n , there are n different merging operations. If each one requires the computer to auto-initialize an array of length n , then the overall sorting algorithm becomes an $O(N^2)$ algorithm. That is considerably slower than the $O(N \log N)$ behavior that we expect for a merge sort algorithm.

The authors made this mistake because C and C++ don't auto-initialize arrays. That means that their code ran fast in those languages, and when they tested their code they found that it behaved as expected. They apparently translated their code into Java without retesting whether it was fast. More than one textbook author has made this mistake, but out of respect for our fellow authors (and recognizing that we make mistakes ourselves), we won't name any names!

Chapter Summary

Searching is the task of attempting to find a particular target value in a collection or array.

Sorting is the task of arranging the elements of a list or array into a natural ordering.

Java's class libraries contain several methods for searching and sorting arrays and lists, such as `Arrays.binarySearch` and `Collections.sort`.

A `Comparator` object describes a customized way to compare objects, enabling arrays or lists of these objects to be searched and sorted in many orders.

Empirical analysis is the technique of running a program or algorithm to determine its runtime. Algorithm analysis is the technique of examining an algorithm's code or pseudocode to make inferences about its complexity.

Algorithms are grouped into complexity classes, which are often described using big-Oh notation such as $O(N)$ for a linear algorithm.

Sequential search is an $O(N)$ searching algorithm that looks at every element of a list until it finds the target value.

Binary search is an $O(\log N)$ searching algorithm that operates on a sorted dataset and successively eliminates half of the data until it finds the target element.

Selection sort is an $O(N^2)$ sorting algorithm that repeatedly finds the smallest unprocessed element of the array and moves it to the frontmost remaining slot of the array.

Merge sort is an $O(N \log N)$ sorting algorithm, often implemented recursively, that successively divides the input dataset into two halves, recursively sorts the halves, and then merges the sorted halves into a sorted whole.

Self-Check Problems

Section 13.1: Searching and Sorting in the Java Class Libraries

1. Describe two ways to search an unsorted array of `String` objects using the Java class libraries.
2. If you perform a binary search on an array of one million integers, which of the following is closest to the number of elements that the search algorithm will need to examine?
 - a. all 1,000,000 of the integers
 - b. roughly 3/4 (750,000) of the integers
 - c. roughly half (500,000) of the integers
 - d. roughly 1/10 (100,000)
 - e. less than 1% (10,000 or fewer)
3. Should you use a sequential or binary search on an array of `Point` objects, and why?
4. Under what circumstances can the `Arrays.binarySearch` and `Collections.binarySearch` methods be used successfully?
5. In what order does the `Collections.sort` method arrange a list of strings? How could you arrange them into a different order?
6. Why wouldn't the `Collections.sort` method work when used on a list of `Point` objects? How can you make it so that the sort method can be used on `Points` or any other type of objects?

7. The following `Comparator` class is attempting to arrange `BankAccount` objects by account name, breaking ties by account balance. But the code has some syntax errors and some logic errors. What is wrong with the code? How would you correct it?

```
1 import java.util.*;
2 public class AccountComparator extends Comparator {
3     public int compareTo(BankAccount account2) {
4         if
5             (!this.getName().equals(account2.getName())) {
6             return
7                 this.getName().compareTo(account2.getName());
8         } else {
9             return this.getBalance() -
10                account2.getBalance();
11         }
12     }
13 }
```

8. In this section, we wrote a `LengthComparator` that would allow a list or array of strings to be sorted in ascending order by length. How could we easily sort the collection in descending order by length, from longest to shortest, without modifying the code of `LengthComparator`?

Section 13.2: Program Complexity

9. Approximate the runtime of the following code fragment, in terms of n :

```
int sum = 0;
int j = 1;
while (j <= n) {
    sum++;
    j = j * 2;
}
```

10. Approximate the runtime of the following code fragment, in terms of n :

```
int sum = 0;
for (int j = 1; j < n; j++) {
    sum++;
    if (j % 2 == 0) {
        sum++;
    }
}
```

11. Approximate the runtime of the following code fragment, in terms of n :

```

int sum = 0;
for (int i = 1; i <= n * 2; i++) {
    for (int j = 1; j <= n; j++) {
        sum++;
    }
}
for (int j = 1; j < 100; j++) {
    sum++;
    sum++;
}

```

12. Approximate the runtime of the following code fragment, in terms of n :

```

int sum = 0;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j += 2) {
        sum += 4;
    }
}
for (int k = -50; k <= -1; k++) {
    sum--;
}

```

13. Approximate the runtime of the following code fragment, in terms of n :

```
int sum = 0;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= 1000000; j++) {
        sum += 10;
    }
}
sum += 9999;
```

14. Determine the complexity classes of the algorithms that could be used to perform the following tasks:
- Finding the average of the numbers in an array of integers
 - Finding the closest distance between any pair of points in an array of `Points`
 - Finding the maximum value in an array of real numbers
 - Counting the median length of the `Strings` in an array
 - Raising an integer to a power—for example, A^B
 - Examining an array of `Points` to see how many trios of points are colinear—that is, how many groups of three points could be connected by a straight line
 - Counting the number of lines in a file
 - Determining whether a given integer representing a year stores a leap year (a year divisible by 4, but not divisible by 100 unless also divisible by 400)

15. Suppose an algorithm takes exactly the given number of statements for each value below, in terms of an input size N . Give a tight big-Oh bound for each algorithm, representing the closest complexity class for that algorithm based on that runtime.

- a. $\frac{1}{2}N \log N + \log N$
- b. $N^2 - (N + N \log N + 1000)$
- c. $N^2 \log N + 2N$
- d. $\frac{1}{2}(3N + 5 + N)$
- e. $(2N + 5 + N^4)/N$
- f. $\log(2^N)$
- g. $N! + 2N$

Section 13.3: Implementing Searching and Sorting Algorithms

16. What is the runtime complexity class of a sequential search on an unsorted array? What is the runtime complexity class of the modified sequential search on a sorted array?
17. Why does the binary search algorithm require the input to be sorted?
18. How many elements (at most) does a binary search examine if the array contains 60 elements?
19. What indexes will be examined as the middle element by a binary search for the target value `8` when the search is run on the following input arrays? What value will the binary search algorithm return?
 - a. `int[] numbers = {1, 3, 6, 7, 8, 10, 15, 20, 30};`
 - b. `int[] numbers = {1, 2, 3, 4, 5, 7, 8, 9, 10};`
 - c. `int[] numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9};`
 - d. `int[] numbers = {8, 9, 12, 14, 15, 17, 19, 25, 31};`
20. What indexes will be examined as the middle element by a binary search for the target value `8` when the search is run on the following input array? Notice that the input array isn't in sorted order. What can you say about the binary search algorithm's result?

```
int[] numbers = {6, 5, 8, 19, 7, 35, 22, 11, 9};
```

21. Consider the following sorted array of integers. When a binary search is performed on this array for each of the following integer values, what indexes are examined in order? What result value is returned?

```
// index      0    1    2    3    4    5    6    7    8    9
10  11  12  13  14
int[] numbers = {-1,  3,  5,  8, 15, 18, 22, 39, 40, 42,
50, 57, 71, 73, 74};
```

- a. 42
- b. 11
- c. 74
- d. 30

22. Consider the following sorted array of integers. When a binary search is performed on this array for each of the following integer values, what indexes are examined in order? What result value is returned?

```
// index      0    1    2    3    4    5    6    7    8    9
10  11  12  13
int[] numbers = {-30, -9, -6, -4, -2, -1,  0,  2,  4, 10,
12, 17, 22, 30};
```

- a. -5
- b. 0
- c. 11
- d. -100

23. What modifications would you have to make to the `selectionSort` method to cause it to sort an array of `double` values rather than one of integer values?

24. Consider the following array:

```
int[] numbers = {29, 17, 3, 94, 46, 8, -4, 12};
```

After a single pass of the selection sort algorithm (a single swap), what would be the state of the array?

- a. {-4, 29, 17, 3, 94, 46, 8, 12}
- b. {29, 17, 3, 94, 46, 8, 12}
- c. {-4, 29, 17, 3, 94, 46, 8, -4, 12}
- d. {-4, 17, 3, 94, 46, 8, 29, 12}
- e. {3, 17, 29, 94, -4, 8, 46, 12}

25. Trace the execution of the selection sort algorithm as shown in this section when run on the following input arrays. Show each element that will be selected by the algorithm and where it will be moved, until the array is fully sorted.

- a. {29, 17, 3, 94, 46, 8, -4, 12}
- b. {33, 14, 3, 95, 47, 9, -42, 13}

c. {7, 1, 6, 12, -3, 8, 4, 21, 2, 30, -1, 9}

d. {6, 7, 4, 8, 11, 1, 10, 3, 5, 9}

Section 13.4: Case Study: Implementing Merge Sort

26. How many calls on the `mergeSort` method are generated by a call to sort a list of length 32?

27. Consider the following array of `int` elements:

```
int[] numbers = {7, 2, 8, 4, 1, 11, 9, 5, 3, 10};
```

- Show the state of the elements after five passes of the outermost loop of selection sort have occurred.
- Show a trace that is two levels deep of the merge sort algorithm. Show the splitting of the overall array, plus one level of the recursive calls.

28. Consider the following array of `int` elements:

```
int[] numbers = {7, 1, 6, 12, -3, 8, 4, 21, 2, 30, -1,  
9};
```

- Show the state of the elements after five passes of the outermost loop of selection sort have occurred.
- Show a trace that is two levels deep of the merge sort algorithm. Show the splitting of the overall array, plus one level of the recursive calls.

29. Which one of the following statements about sorting and big-Oh is true?
- a. Selection sort can sort an array of integers in $O(N)$ time.
 - b. Merge sort achieves an $O(N \log N)$ runtime by dividing the array in half at each step and then recursively sorting and merging the halves back together.
 - c. Merge sort runs faster than selection sort because it is recursive, and recursion is faster than loops.
 - d. Selection sort runs in $O(N)$ time if the array is already sorted to begin with, or $O(N^2)$ if it is not.
 - e. Sorting algorithms that rely on comparing elements can only be used with type `int`, because values from other types of data cannot be compared to each other.
30. Trace the complete execution of the merge sort algorithm when called on each array below. Show the sub-arrays that are created by the algorithm and show the merging of sub-arrays into larger sorted arrays.
- a. `{29, 17, 3, 94, 46, 8, -4, 12}`
 - b. `{6, 5, 3, 7, 1, 8, 4, 2}`
 - c. `{33, 14, 3, 95, 47, 9, -42, 13}`

Exercises

1. Suppose the following array has been declared:

```
// index      0  1  2  3  4  5  6  7  8  9
int[] list = {-2, 8, 13, 22, 25, 25, 38, 42, 51, 103};
```

What indexes will be examined as the middle element by a binary search for each of the following target values? What value will be returned?

- a. 103
- b. 30
- c. 8
- d. -1

2. Suppose the following array has been declared:

```
// index      0  1  2  3  4  5  6  7  8  9
10 11
int[] numbers = {-1, 3, 5, 8, 15, 18, 22, 39, 40, 42,
50, 57};
```

What indexes will be examined as the middle element by a binary search for each of the following target values? What value will be returned?

- a. 13
- b. 39
- c. 50
- d. 2

3. Suppose the following array has been declared:

```
// index      0   1   2   3   4   5   6   7   8   9
10  11  12  13  14
int[] numbers = {0,  0,  5, 10, 15, 40, 55, 60, 65, 70,
80, 85, 90, 95, 300};
```

What indexes will be examined as the middle element by a binary search for each of the following target values? What value will be returned?

- a. 65
- b. 9
- c. 90
- d. 147

4. To which complexity class does the following algorithm belong? Consider N to be the length or size of the array or collection passed to the method. Explain your reasoning.

```
public static int[] mystery1(int[] list) {  
    int[] result = new int[2 * list.length];  
    for (int i = 0; i < list.length; i++) {  
        result[2 * i] = list[i] / 2 + list[i] % 2;  
        result[2 * i + 1] = list[i] / 2;  
    }  
    return result;  
}
```

5. To which complexity class does the following algorithm belong?

```
public static void mystery2(int[] list) {  
    for (int i = 0; i < list.length / 2; i++) {  
        int j = list.length - 1 - i;  
        int temp = list[i];  
        list[i] = list[j];  
        list[j] = temp;  
    }  
}
```

6. To which complexity class does the following algorithm belong?

```

public static void mystery3(List<String> list) {
    for (int i = 0; i < list.size() - 1; i += 2) {
        String first = list.remove(i);
        list.add(i + 1, first);
    }
}

```

7. To which complexity class does the following algorithm belong?

```

public static void mystery4(List<String> list) {
    for (int i = 0; i < list.size() - 1; i += 2) {
        String first = list.get(i);
        list.set(i, list.get(i + 1));
        list.set(i + 1, first);
    }
}

```

8. Write the state of the elements of each of the following arrays after each pass of the outermost loop of the selection sort algorithm has occurred (after each element is selected and moved into place).

```

int[] numbers1 = {63, 9, 45, 72, 27, 18, 54, 36};

```

```
int[] numbers2 = {37, 29, 19, 48, 23, 55, 74, 12};
```

9. Using the same arrays from the previous problem, trace the complete execution of the merge sort algorithm when called on each array. Show the subarrays that are created by the algorithm and show the merging of subarrays into larger sorted arrays.
10. Write the state of the elements of each of the following arrays after each pass of the outermost loop of the selection sort algorithm has occurred (after each element is selected and moved into place).

```
int[] numbers3 = {8, 5, -9, 14, 0, -1, -7, 3};  
int[] numbers4 = {15, 56, 24, 5, 39, -4, 27, 10};
```

11. Using the same arrays from the previous problem, trace the complete execution of the merge sort algorithm when called on each array. Show the subarrays that are created by the algorithm and show the merging of subarrays into larger sorted arrays.
12. Write the state of the elements of each of the following arrays after each pass of the outermost loop of the selection sort algorithm has occurred (after each element is selected and moved into place).

```
int[] numbers5 = {22, 44, 11, 88, 66, 33, 55, 77};
```

```
int[] numbers6 = {-3, -6, -1, -5, 0, -2, -4, -7};
```

13. Using the same arrays from the previous problem, trace the complete execution of the merge sort algorithm when called on each array. Show the subarrays that are created by the algorithm and show the merging of subarrays into larger sorted arrays.
14. Write code to read a dictionary from a file, then prompt the user for two words and tell the user how many words in the dictionary fall between those two words. Here is a sample run of the program:

```
Type two words: goodbye hello
```

```
There are 4418 words between goodbye and hello
```

Use the binary search algorithm in your solution.

15. Write a `Comparator` that compares `Point` objects by their distance from the origin of (0, 0). Points that are closer to the origin are considered to come before those which are further from the origin.
16. Write a `Comparator` that compares `String` objects by the number of words they contain. Consider any nonwhitespace string of characters to be a word. For example, “hello” comes before “I see”, which comes before “You can do it”.
17. Write a `Comparator` that compares `String` objects of a particular format. Each string is of a form such as `"123456"`

`Seattle, WA"`, beginning with a numeric token that is followed by additional text tokens. Your job is to treat the first tokens as integers and compare them in numerical order. You cannot simply compare them by using the strings' `compareTo` method, since it would treat the numbers as text and not as integers. For example, `"276453 Helena, MT"` is greater than `"9847 New York, NY"`. Use a `Scanner` to tokenize the strings while comparing them.

18. Write a modified version of the selection sort algorithm that selects the largest element each time and moves it to the end of the array, rather than selecting the smallest element and moving it to the beginning. Will this algorithm be faster than the standard selection sort? What will its complexity class (big-Oh) be?
19. Write a modified “dual” version of the selection sort algorithm that selects both the largest and smallest elements on each pass and moves each of them to the appropriate end of the array. Will this algorithm be faster than the standard selection sort? What predictions would you make about its performance relative to the merge sort algorithm? What will its complexity class (big-Oh) be?
20. Implement an algorithm to shuffle an array of numbers or objects. The algorithm for shuffling should be the following:

```
for (each index i) {  
    choose a random index j where j >= i.
```

```
    swap the elements at indexes i and j.  
}
```

(The constraint about `j` being greater than or equal to `i` is actually quite important, if you want your shuffling algorithm to shuffle fairly. Why?)

21. Implement a “bogus” sorting algorithm called *bogo sort* that uses your shuffling algorithm from the previous exercise to sort an array of numbers. The bogo sort algorithm is the following:

```
while (array is not sorted) {  
    shuffle array.  
}
```

Obviously, this is not a very efficient sorting algorithm, but it eventually does shuffle the array into order if you let it run long enough. Try running it on a very small array, such as 8 or 10 elements, to examine its runtime. What is your best guess about the complexity class (big-Oh) of this silly algorithm?

Programming Projects

1. Write a program that reads a series of input lines and sorts them into alphabetical order, ignoring the case of words. The program should use the merge sort algorithm so that it efficiently sorts even a large file.
2. Perform a “Sort Detective” challenge to run several sorting algorithms without knowing which is which. Try to figure out which sorting algorithm is which on the basis of the runtime and characteristics of each algorithm. Search the web for “sort detective” for more ideas on such a project.
3. Write a program that processes a data file of students’ course grade data. The data arrive in random order; each line stores information about a student’s last name, first name, student ID number, grade as a percentage, and letter grade. For example, here are a few lines of data:

Smith	Kelly	438975	98.6	A
Johnson	Gus	210498	72.4	C
Reges	Stu	098736	88.2	B
Smith	Marty	346282	84.1	B
Reges	Abe	298575	78.3	C

Your program should be able to sort the data by any of the columns. Use `Comparators` to achieve the sort orderings. Make

the data sortable by last name, student ID, and grade percentage in ascending and descending order. For example, here are the lines of student data sorted a few different ways:

```
Student data, by last name:
```

Johnson	Gus	210498	72.4	C
Reges	Stu	098736	88.2	B
Reges	Abe	298575	78.3	C
Smith	Kelly	438975	98.6	A
Smith	Marty	346282	84.1	B

```
Student data, by student ID:
```

Reges	Stu	098736	88.2	B
Johnson	Gus	210498	72.4	C
Reges	Abe	298575	78.3	C
Smith	Marty	346282	84.1	B
Smith	Kelly	438975	98.6	A

4. Write a program that discovers all anagrams of all words listed in an input file that stores the entries in a large dictionary. An anagram of a word is a rearrangement of its letters into a new legal word. For example, the anagrams of “share” include “shear”, “hears”, and “hares”. Assume that you have a file available to you that lists many words, one per line. Your program should first read in the dictionary file and sort it, but instead of sorting in alphabetical order it should sort according to each word’s canonical form. The canonical form

of a word contains the same letters as the original, but in sorted order. Thus, the canonical form of “computer” is “cemoprut”, and the canonical form of “program” is “agmoprr”. When your dictionary file is sorted, the word “program” would be placed before the word “computer”, because its canonical form comes first in alphabetical order. Write code to retrieve a word’s canonical form and a `Comparator` that compares words by using their canonical forms.